

GOETHE-UNIVERSITÄT FRANKFURT AM MAIN

FACHBEREICH INFORMATIK

Instruction Robustness in RAG/GraphRAG Agent Pipelines

Bachelor Thesis

Name: Christian Block

Matriculation Number: 6522508

First examiner: Prof. Dr. Visvanathan Ramesh

Date of Submission:

Erklärung zur Abschlussarbeit

gemäß § 35, Abs. 16 der Ordnung für den Bachelorstudiengang Informatik vom 17. Juni 2019:

Hiermit erkläre ich (Nachname, Vorname) **Block, Christian**, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst habe. Ich bestätige außerdem, dass die vorliegende Arbeit nicht, auch nicht auszugsweise, für eine andere Prüfung oder Studienleistung verwendet wurde. Zudem versichere ich, dass alle eingereichten schriftlichen gebundenen Versionen meiner vorliegenden Bachelorarbeit mit der digital eingereichten elektronischen Version meiner Bachelorarbeit übereinstimmen.

Frankfurt am Main, den DATUM

Unterschrift der/des Studierenden

Abstract

Retrieval-Augmented Generation (RAG) systems combine information retrieval with language generation, letting a language model condition its output on external, updatable evidence instead of only on what it learned during training. Within such a system, the system prompt specifies the instructions, constraints, and output format the generator should follow. Because the system prompt is written in natural language, its effect on system behaviour is hard to predict and hard to verify, and prior work on prompt sensitivity in language models has studied this almost entirely outside retrieval-augmented settings. This thesis addresses that gap by asking to what extent systematic changes in the system prompt alter the behaviour and outcomes of a RAG system when the user input, knowledge base, retrieval mechanism, and underlying language model are held constant.

The thesis develops a formal model of a RAG system based on the probabilistic formulation of Lewis et al. (2020), in which a retriever assigns probabilities $p_\eta(z \mid q, D)$ to candidate documents and a generator produces the answer token by token, with the system prompt added as an explicit conditioning variable $p_\theta(y_i \mid q, z, y_{1:i-1}, S)$ and modelled as a set of checkable requirements Σ written in a wording φ , so that $S = \rho(\Sigma, \varphi)$. Five prompt conditions are defined: a canonical baseline and four controlled interventions, three of which change only the wording φ while holding the requirements fixed, and one that changes the requirement set itself by varying modal force.

Changing the system prompt is treated as an intervention on that model, and what counts as a good answer is defined before any measurement is introduced, as four requirements an answer must meet at once: that it be grounded in the retrieved evidence, relevant to the question asked, correct against the reference answer, and obedient to the instructions it was given. The knowledge base and the query set are drawn from established question-answer datasets rather than constructed for the experiment, which makes the measurement reproducible and its results comparable with published work. The system prompt is the only component that is manipulated: the language model, the embedding model, the vector index, the chunking strategy and the retrieval depth are all held fixed, so that the passages reaching the generator are identical in every condition and any difference in the output originates at the generator alone. The quality of the retrieved evidence is varied across four levels, from ideal through corrupted to absent, as a controlled robustness factor rather than as a second treatment, so that the effect of the prompt can be separated from what the language model already knows. Generations are repeated to account for decoding randomness, and four generation-stage measures are evaluated under a pre-specified, corrected statistical analysis plan with one main result; retrieval-stage measures are excluded by construction, since the system prompt enters the pipeline downstream of retrieval and cannot alter what is retrieved. The result is a controlled methodology for isolating the effect of a RAG system prompt on system behaviour, established for the standard single-shot architecture, which is the configuration in which that effect is identifiable and the baseline from which graph-structured and agentic pipelines depart.

Zusammenfassung

Retrieval-Augmented-Generation-Systeme (RAG-Systeme) verbinden Informationsretrieval mit Sprachgenerierung und ermöglichen es einem Sprachmodell, seine Ausgabe auf externe, aktualisierbare Evidenz zu stützen, anstatt sich ausschließlich auf während des Trainings erlernte Parameter zu verlassen. Innerhalb eines solchen Systems legt der System-Prompt die Anweisungen, Einschränkungen und Ausgabeanforderungen fest, denen der Generator folgen soll. Da der System-Prompt in natürlicher Sprache formuliert ist, lässt sich seine Wirkung auf das Systemverhalten nur schwer vorhersagen und überprüfen; bisherige Arbeiten zur Prompt-Sensitivität von Sprachmodellen wurden fast ausschließlich außerhalb von Retrieval-Augmented-Settings durchgeführt. Die vorliegende Arbeit adressiert diese Lücke, indem sie untersucht, in welchem Ausmaß systematische Änderungen des System-Prompts das Verhalten und die Ergebnisse eines RAG-Systems verändern, wenn Nutzereingabe, Wissensbasis, Retrieval-Mechanismus und zugrunde liegendes Sprachmodell konstant gehalten werden.

Die Arbeit entwickelt ein formales Modell eines RAG-Systems auf Grundlage der probabilistischen Formulierung von Lewis et al. (2020), in der ein Retriever den Kandidatendokumenten Wahrscheinlichkeiten $p_\eta(z \mid q, D)$ zuweist und ein Generator die Antwort Token für Token erzeugt, wobei der System-Prompt als explizite Bedingungsvariable $p_\theta(y_i \mid q, z, y_{1:i-1}, S)$ eingeführt und als eine Menge überprüfbarer Anforderungen Σ modelliert wird, die in einer Formulierung φ ausgedrückt werden, sodass $S = \rho(\Sigma, \varphi)$ gilt. Es werden fünf Prompt-Bedingungen definiert: eine kanonische Baseline sowie vier kontrollierte Interventionen, von denen drei ausschließlich die Formulierung φ verändern, während die Anforderungen konstant gehalten werden, sowie eine weitere, die durch Variation der modalen Kraft die Anforderungsmenge selbst verändert.

Die Änderung des System-Prompts wird als Intervention auf dieses Modell behandelt, und was als gute Antwort gilt, wird festgelegt, bevor eine Messung eingeführt wird: als vier Anforderungen, die eine Antwort zugleich erfüllen muss, nämlich in der abgerufenen Evidenz verankert, für die gestellte Frage relevant, gegenüber der Referenzantwort korrekt und den erteilten Anweisungen gehorsam zu sein. Wissensbasis und Anfragemenge stammen aus etablierten Frage-Antwort-Datensätzen und werden nicht eigens für das Experiment konstruiert, wodurch die Messung reproduzierbar und mit publizierten Ergebnissen vergleichbar wird. Der System-Prompt ist die einzige manipulierte Komponente: Sprachmodell, Embedding-Modell, Vektorindex, Chunking-Strategie und Retrieval-Tiefe werden konstant gehalten, sodass in jeder Bedingung dieselben Passagen den Generator erreichen und jede Abweichung in der Ausgabe allein am Generator entsteht. Die Qualität der abgerufenen Evidenz wird über vier Stufen variiert, von idealer über verfälschte bis hin zu fehlender Evidenz, jedoch als kontrollierter Robustheitsfaktor und nicht als zweite Behandlung, sodass die Wirkung des Prompts vom parametrischen Wissen des Sprachmodells getrennt werden kann. Generierungen werden wiederholt, um die Stochastizität der Dekodierung zu berücksichtigen, und vier generierungsseitige Metriken werden im Rahmen eines vorab festgelegten, korrigierten statistischen Analyseplans mit einem Hauptergebnis ausgewertet; retrieval-seitige Metriken bleiben konstruktionsbedingt ausgeschlossen, da der System-Prompt die Pipeline erst nach dem Retrieval betritt. Das Ergebnis ist eine kontrollierte Methodik zur Isolierung der Wirkung eines RAG-System-Prompts auf

das Systemverhalten, etabliert für die standardmäßige Single-Shot-Architektur, in der dieser Effekt identifizierbar ist und die den Ausgangspunkt für graphbasierte und agentische Pipelines bildet.

Acknowledgments

I want to express my sincere gratitude to Professor Ramesh for his guidance and mentorship throughout this thesis.

Contents

Acronyms	9
1 Introduction	10
1.1 Background	10
1.2 Motivation and Research Objective	10
1.3 Thesis Structure	11
Notation	11
2 Preliminaries	14
2.1 Path towards RAG	14
2.2 Components of RAG	15
2.2.1 Knowledge Base / Vector Database	15
2.2.2 Retriever	16
2.2.3 Generator	17
2.2.4 System Prompt	18
2.3 Retrieval-Augmented Generation and Fine-Tuning	20
2.4 Use-Case of RAG	21
2.5 Advanced RAG Architectures	21
3 Literature Review	24
3.1 Prompt Sensitivity and Instruction-Following in Language Models	24
3.2 Retrieval-Augmented Generation: Architecture, Evaluation, and Failure Modes	25
3.3 Adversarial and Competing Instructions in Retrieved Content	26
3.4 Synthesis and Research Gap	27
4 Formal Model of a RAG System	29
4.1 Modelling Objective	29
4.2 Probabilistic Retrieval	29
4.3 Prompt-Conditioned Generation	30
4.4 Marginalizing over Retrieved Documents	30
4.4.1 RAG-Sequence	30
4.4.2 RAG-Token	30
4.5 The System Prompt: Requirements and Wording	31
4.6 The System Prompt as an Intervention	33
4.7 The Causal Effect	33
4.8 Grounding the Knowledge Base in Established Datasets	34
4.9 What Counts as a Good Answer	35
5 Experimental Setup	38
5.1 Goal and Design	38
5.2 Knowledge Base and Query Set	40

5.3	Experimental Conditions	42
5.4	Metrics	43
5.4.1	The Judge	45
5.5	Procedure and Analysis	45
5.5.1	Provenance of the Prompt Wordings	46
5.6	Limitations	47
6	Results and Discussion	48
7	Conclusion	49
7.1	Contribution	49
7.2	Future Work	49

Acronyms

AI Artificial Intelligence.

LLM Large Language Model.

RAG Retrieval-Augmented Generation.

1 Introduction

1.1 Background

Retrieval-Augmented Generation (RAG) systems combine information retrieval with language generation. This design solves a problem with purely parametric large language models (LLMs): a model’s knowledge is fixed at training time and cannot be updated without retraining or fine-tuning it. Lewis et al. (2020) introduced RAG to address this problem by combining a pretrained retriever with a pretrained sequence-to-sequence generator, so the model can base its output on external, updatable evidence instead of only on what it learned during training. In a RAG system, a user query is first used to retrieve relevant information from an external knowledge base, typically a vector database. This retrieved information is then given to a language model, together with instructions for generating the final response.

The system prompt connects the intended behaviour of the system to its observable output. Unlike the retrieved evidence, which depends on the query and the knowledge base, the system prompt is written on purpose: it specifies how the model should behave, which instructions it should follow, which constraints it should respect, and which output format it should produce. Because the system prompt is written in natural language rather than a formal specification, its effect on behaviour is hard to predict and hard to check in advance. This is why its influence on both intended behaviour and actual output needs to be studied through controlled experiments.

1.2 Motivation and Research Objective

Understanding how a RAG system responds to its system prompt matters in practice. Whether a system reliably follows its instructions decides whether its behaviour can be trusted once deployed, especially when the same retrieval and generation components are reused across many applications with only the system prompt changing. This raises the question of *how systematic changes in the system prompt can be shown to alter the behaviour and outcomes of a RAG system*.

At the same time, isolating the effect of the system prompt is hard. Its influence overlaps with the influence of the retrieved evidence, the language model itself, and even the exact wording of the instructions. All of these must be controlled before any change in behaviour can be attributed to the system prompt rather than to something else. Prior work has shown that small, seemingly harmless changes to the wording of an instruction can cause large changes in model behaviour (Salinas and Morstatter, 2024), and that a system prompt’s authority over the model is not guaranteed once the user turn works against it (Mu et al., 2025). Most of that work varies the instruction the user supplies rather than the standing system prompt, a distinction Section 3.1 draws out and one this thesis is built to address. This is why the present thesis treats the system prompt as a controlled experimental variable, not just a minor setting.

This thesis therefore focuses on one central research question: *To what extent do systematic changes in the system prompt alter the behaviour and outcomes of a RAG system when the user input, knowledge base, retrieval mechanism, and underlying language model are held constant?* Answering

it requires two things: a formal model of the RAG system precise enough to isolate the system prompt as a controlled parameter, and an experimental design that holds every other source of variation fixed. The rest of this thesis builds both. Both are established for the standard single-shot retrieval-augmented architecture, and that restriction is deliberate. As Section 2.5 argues, it is the configuration in which the system prompt reaches the answer along a single path and its effect is therefore identifiable, whereas graph-structured and agentic pipelines allow the prompt to shape retrieval as well, so that an effect measured inside them could not be decomposed into a prompt component and a routing component. What is established here is accordingly a controlled baseline for those richer pipelines rather than a claim about them, and extending the design to a graph-structured or agentic setting is left to future work.

1.3 Thesis Structure

This thesis has seven chapters. Chapter 2 covers the background needed for the rest of the work: it traces how retrieval-augmented approaches to language generation developed, and introduces the components of a RAG system used throughout later chapters. Chapter 3 reviews the relevant literature in three parts, prompt sensitivity in language models, retrieval-augmented generation specifically, and adversarial instructions delivered through retrieved content, and identifies the resulting research gap. Chapter 4 builds a formal model of the RAG system used in this thesis: the probabilistic formulation of Lewis et al. (2020), extended so that the system prompt becomes an explicit variable of the generator, together with the grounding of the knowledge base in established question-answer datasets and a definition of what counts as a good answer. Chapter 5 describes the experimental setup: how the corpus and query set are assembled from those datasets, which conditions the system is run under, how each part of that definition is measured, and which estimators are used for the criterion defined in Chapter 4. Chapter 6 presents and discusses the results of the experiments. Chapter 7 concludes the thesis by summarising the findings and outlining directions for future work.

Notation

Throughout this thesis, the following notation is used. It follows the probabilistic formulation of Lewis et al. (2020) for the system itself and the intervention calculus of Pearl (2009) for the causal statements, and every symbol used in the later chapters is listed here rather than introduced again where it first appears.

The system and its components

- $q \in \mathcal{Q}$ denotes a user query, drawn from the space of possible queries $\mathcal{Q}$, and $P_{\mathcal{Q}}$ denotes the distribution over that space from which queries are drawn.
- $D = \{d_1, d_2, \dots, d_N\} \in \mathcal{D}$ denotes the knowledge base, where each d_i is a document, N is the number of documents, and $\mathcal{D}$ is the space of possible knowledge bases.
- $S \in \mathcal{S}$ denotes a system prompt, drawn from the space of possible system prompts $\mathcal{S}$. It is the standing instruction supplied in the system role, fixed across requests, and is distinct from the query q , which the user supplies anew with each request.
- $C \in \mathcal{C}$ denotes the retrieved context supplied to the language model.

- $y = (y_1, \dots, y_L) \in \mathcal{Y}$ denotes the generated system output, a sequence of L tokens drawn from the space of possible responses $\mathcal{Y}$; $y_{1:i-1}$ denotes the tokens produced before position i .
- R and G denote the retrieval and generation functions of the pipeline, and χ the chunking strategy by which documents are divided into retrievable units.
- $v \in \mathcal{V}$ denotes a token of the vocabulary $\mathcal{V}$, u_v the score the generator assigns it, and $T > 0$ the decoding temperature.

The probabilistic model

- $z \in D$ denotes a single retrieved document, and $\mathcal{Z}_K(q, D)$ the K documents with the highest retrieval probability.
- $p_\eta(z \mid q, D)$ denotes the retrieval distribution with parameters η , and $p_\theta(y_i \mid q, z, y_{1:i-1}, S)$ the generator with parameters θ .
- $p_{\text{RAG}}(y \mid q, D, S)$ denotes the distribution over outputs obtained by marginalising the generator over the retrieved documents.
- Y denotes the output viewed as a random variable, $M(Y)$ a measured property of it, and ε the randomness introduced by decoding.

The system prompt and its variation

- $\Sigma = \{\sigma_1, \dots, \sigma_m\}$ denotes the set of requirements a system prompt states, where each σ_k is a property an answer either has or does not have, so that $\sigma_k(y) \in \{0, 1\}$ is checkable on any output.
- φ denotes the wording in which those requirements are expressed, and $S = \rho(\Sigma, \varphi)$ denotes the resulting system prompt.
- $S_{b,0}, \dots, S_{b,4}$ denote the five prompt conditions of baseline $b \in \{A, B, C\}$, fifteen in all, with $S_{b,0}$ the canonical form of that baseline. The first four share Σ_b and differ in wording; the fifth is built from a modified requirement set Σ'_b .

Causal intervention

- $\text{do}(S = S_j)$ denotes the intervention that sets the system prompt to condition j , as distinct from observing it to take that value.
- τ_j denotes the average causal effect of condition S_j relative to the baseline S_0 , taken over the query distribution P_Q , and δ_j the interaction contrast between two evidence levels.
- $\mathbb{E}[\cdot]$ denotes expectation, and $\hat{\cdot}$ marks a quantity estimated from the sample rather than defined in the model.

Evidence and evaluation

- Φ denotes the facts the dataset annotations hold true, Φ_y the facts an answer asserts, Φ_C the facts contained in the retrieved context, and a_q the reference answer annotated for query q .
- $\mathcal{Y}^*(q, C)$ denotes the set of acceptable answers, $n_k(y)$ the number of times an answer breaks requirement k , λ_k the weight attached to that breach, and $\ell(y)$ the resulting cost of an answer.

Experimental design

- c_{ik} denotes the k -th context given to query q_i , with c^{gold} , c^{ret} , c^{corr} and $c^{\emptyset}$ the four evidence levels: annotated supporting passages, the genuine retriever output, corrupted passages, and no context at all.
- $\mathcal{Q}_{\text{direct}}$ and $\mathcal{Q}_{\text{multi-hop}}$ denote the two query categories, and N_Q the size of the query set.
- π_i denotes the share of runs in which query q_i is answered correctly with no context at all.
- $\hat{\mathcal{R}}(S_j, c^k)$ denotes the empirical risk, the average cost of the answers produced in one cell of the design, and $\hat{\tau}_{jk}$ the sample estimate of τ_j within evidence level k .
- The indices are $i \in \{1, \dots, N_Q\}$ over queries, $j \in \{0, \dots, 4\}$ over prompt conditions, $k \in \{1, \dots, 4\}$ over evidence levels, and $r \in \{1, \dots, n\}$ over the n repetitions of each cell.

2 Preliminaries

Chapter 1 established that the system prompt is the interface through which a RAG system’s intended behaviour is specified, and asked how systematically varying this interface changes system behaviour and outcomes. Before that question can be answered formally, the components of a RAG system and the reasons behind its design need to be introduced. This chapter covers that background: Section 2.1 traces how retrieval-augmented approaches to language generation developed, Section 2.2 introduces the components of a RAG system used throughout later chapters, and Section 2.3 discusses typical use cases and the failure modes they expose.

2.1 Path towards RAG

The need to store knowledge outside of any single memory, and to retrieve it again on demand, predates computing by a wide margin. Inca administrators recorded census and tax data in knotted cords, quipu, that only specially trained officials could encode or decode, separating the record itself from the person consulting it. Centuries earlier, the Library of Alexandria’s cataloger Callimachus organised its holdings by author and subject in the Pinakes, one of the first attempts to make a large external collection searchable rather than merely stored. Bush (1945) made the need for machine-assisted retrieval explicit, proposing a device that could store a person’s library and retrieve from it through associative links between related items rather than a single fixed order. Codd (1970) gave this idea a rigorous computational form, treating a knowledge store as a formally queryable structure, and Salton et al. (1975) extended retrieval from exact structured queries to similarity ranking over unstructured text, the direct ancestor of the embedding-based retrieval described later in this chapter.

This same need, a mechanism to retrieve from an external store rather than to remember everything internally, carried directly into question-answering systems, which did not begin with language models either. Early systems relied on hand-crafted rules and pattern matching over the input question, mapping recognised syntactic patterns directly onto retrieval or answer-generation procedures (Pandya and Bhatt, 2021). As statistical and later neural methods replaced these rule-based components, the dominant approach shifted toward learning the mapping from question to answer directly from data, culminating in large language models that generate answers from parameters learned over large text corpora rather than from any explicit rule set.

Large language models generate text using only parametric knowledge, that is, information encoded in their weights during training (Wang et al., 2024). This has two problems. First, a model’s knowledge is fixed at the point its training data was collected: any event, fact, or document from after that point is unknown to the model unless it is retrained or fine-tuned, which is costly and has to be repeated every time knowledge changes. Second, purely parametric models are prone to hallucination: they can generate fluent, plausible-sounding statements that no evidence actually supports, because nothing in their design forces an output to trace back to a verifiable source.

Both problems are the reason to give a language model access to an external, updatable source of evidence at inference time, instead of relying only on what is stored in its parameters (Lin et al.,

2025). Lewis et al. (2020) formalised this idea as Retrieval-Augmented Generation. It combines a pretrained neural retriever, which selects relevant passages from a document collection, with a pretrained sequence-to-sequence generator, which conditions its output on those retrieved passages in addition to the input query. This means the same language model can be paired with different or updated document collections without retraining, and a generated response can be grounded in, and in principle traced back to, specific retrieved evidence.

Since it was introduced, RAG has developed beyond this original design. Gao et al. (2023c) describe three stages of this development: Naive RAG, where retrieval and generation run as one fixed pipeline; Advanced RAG, which adds steps such as query reformulation or re-ranking retrieved passages before generation; and Modular RAG, where retrieval, generation, and other steps are separate, swappable components that can be rearranged for different tasks. In all of these variants, retrieval and generation stay separate steps. This is why this thesis treats retrieval and generation as two distinct functions, so their interaction with the system prompt can be studied on its own, as formalised in Chapter 4.

2.2 Components of RAG

A RAG system has four components: the knowledge base that evidence is retrieved from, the retriever that selects relevant evidence for a given query, the generator that produces a response from the query and the retrieved evidence, and the system prompt that specifies how the generator should behave. This section introduces each component in turn; Chapter 4 treats them formally. Figure 2.1 shows how they fit together and which data passes between them.

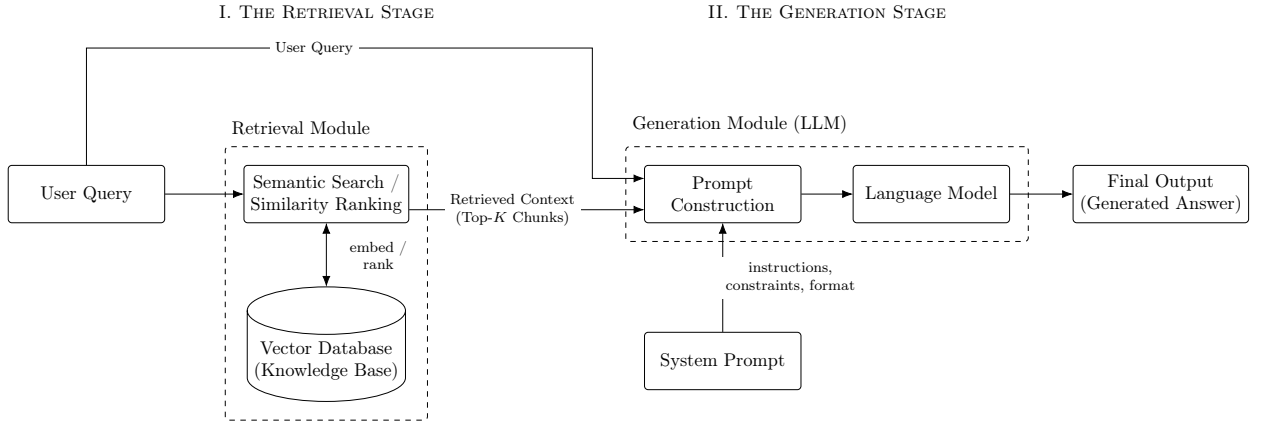


Figure 2.1: Architecture of a RAG system, showing the four components introduced in this section. The retrieval stage matches the user query against a vector database and returns the top- K chunks. The generation stage combines those chunks with the user query and the system prompt into a single prompt and produces the answer.

2.2.1 Knowledge Base / Vector Database

The knowledge base $D = \{d_1, d_2, \dots, d_N\}$ is the collection of documents that evidence is retrieved from. In most modern RAG systems, documents are first split into smaller passages, called chunks. This is needed because a language model can only take in a limited amount of context, and retrieving a whole document would add mostly irrelevant text along with the relevant part. The chunk size is a trade-off: small chunks are precise but may separate a statement from the sentence that qualifies it, while large chunks keep more context but dilute the part that actually matches the query. Chen et al. (2024b) compare retrieval units of different granularity and report that the choice changes

both retrieval quality and the quality of the final answer, which makes chunk size a design parameter rather than an implementation detail.

Each chunk is then mapped to a high-dimensional embedding vector by an embedding model, so that chunks with similar meaning end up close together in the embedding space. Representing text as vectors and measuring relatedness by geometric proximity goes back to the vector space model of Salton et al. (1975); what changed with neural embeddings is that the vector is learned rather than derived from term counts. Similarity between a query vector and a chunk vector is usually measured by the cosine of the angle between them, which ignores vector length and compares direction only.

These embeddings are stored in a vector database, which supports fast nearest-neighbor search over large collections of chunks. Comparing a query against every stored vector is exact but becomes too slow as the collection grows, so vector databases index the vectors and use approximate nearest-neighbor search, which trades a small loss in recall for a large gain in speed. The most widely used index of this kind is the hierarchical navigable small world graph of Malkov and Yashunin (2020), which reaches a nearest neighbour by descending through graph layers of decreasing coarseness instead of scanning the collection. Several systems build on such indexes, and Pan et al. (2024) survey how they differ in indexing, query processing and storage; for this section the relevant difference is how they are deployed. FAISS is a library that is embedded directly in an application and gives fine-grained control over the index structure (Johnson et al., 2021). Chroma is a lightweight database aimed at local development and small deployments. The `pgvector` extension adds vector columns and similarity operators to PostgreSQL, which allows vectors to sit next to existing relational data. Pinecone is a managed service in which indexing and scaling are handled by the provider. For the purposes of this thesis the relevant point is that all four answer the same question, namely which stored vectors lie closest to a query vector, so the choice affects latency, operational cost and reproducibility rather than the logic of the pipeline. The indexing step as a whole, in particular the choice of chunk size and embedding model, decides what evidence is even available to the retriever, regardless of how good the retriever itself is.

2.2.2 Retriever

Given a user query q and a knowledge base D , the retriever selects the chunks that are relevant for answering q . Chapter 4 formalises this as a probability distribution $p_\eta(z \mid q, D)$ over candidate documents. Three approaches are common, and they fail in different ways.

Dense Retrieval

Dense retrieval embeds the query with the same embedding model used for the knowledge base and ranks chunks by the similarity between the query embedding and each chunk embedding. Because the embedding is learned, two passages that express the same content in different words are placed close together, so a query can match a chunk with which it shares no vocabulary. This is the mechanism the original RAG architecture of Lewis et al. (2020) relies on. The weakness is the mirror image of the strength: rare identifiers, product codes, names and numbers carry little semantic signal, and a dense retriever may rank a fluent but irrelevant passage above the one chunk that contains the exact string the query asked about.

Sparse Retrieval

Sparse retrieval scores chunks by term overlap with the query. The standard method is BM25 (Robertson and Zaragoza, 2009), which weights each matching term by how often it occurs in the

chunk, how rare it is across the collection, and how long the chunk is, so that common words contribute little and long chunks are not favoured merely for containing more words. The representation is sparse because a chunk is described by the vocabulary it actually contains rather than by a dense vector. BM25 requires no training and remains a strong baseline, and it handles exactly the cases dense retrieval struggles with, since an exact term match is what it scores. It fails on paraphrase: a query and a chunk that share no terms receive a score of zero regardless of how closely their meanings agree.

Hybrid Search

Since the two methods fail on different queries, many systems run both and combine the results. The difficulty is that the two scores are not on a common scale, so they cannot simply be added. The usual solutions are to normalise the scores before combining them with a weight, or to combine the two rankings rather than the scores. Reciprocal rank fusion (Cormack et al., 2009) is the standard method of the second kind: each chunk is scored by the reciprocal of its rank in each list, so a chunk ranked highly by either method is promoted without the two scores ever being compared directly. Gao et al. (2023c) describe hybrid retrieval as one of the standard refinements distinguishing later RAG systems from the original design.

Retrieval Depth

Whichever method is used, the top- K ranked chunks are returned as the retrieved context. Retrieval quality depends on both how many chunks are retrieved and how precise the ranking is: too few chunks risk missing needed evidence, while too many risk adding irrelevant or conflicting information to the generation step. The order of the chunks also matters, since Liu et al. (2024a) report that answer accuracy depends on where the relevant passage sits within the context. This thesis fixes the retriever, the chunking and K across all experimental conditions, so that changes in the output cannot be traced back to retrieval.

2.2.3 Generator

The generator is the large language model that produces the final response. A large language model is a neural network trained on a large text corpus to predict the next token in a sequence, and it generates text by autoregressive decoding: producing one token at a time, each conditioned on all previous tokens. Its parameters, written θ in Chapter 4, encode what the model learned during training. This is often called parametric memory, and it is fixed once training ends.

Within a RAG pipeline the model is not used primarily as a store of facts. It is used as a reasoning and synthesis component that is conditioned on evidence supplied at inference time: it reads the retrieved chunks, selects the parts that answer the query, resolves references between them, and writes the result as fluent text. This division is the point of the architecture. Facts come from the knowledge base, which can be updated without retraining, while the model contributes language competence and the ability to combine several passages into one answer (Lewis et al., 2020). When the model answers from parametric memory instead of from the retrieved context, the answer may still be fluent and even correct, but it is no longer grounded in evidence that can be checked, which is one route to the unsupported claims documented by Niu et al. (2024).

In practice the query, the retrieved context and the system prompt are not passed to the model separately. They are combined, following a fixed template, into a single input sequence, usually with the system prompt placed first so that it shapes how everything after it is read. Chapter 4

formalises the generator as $p_\theta(y_i \mid q, z, y_{1:i-1}, S)$, the probability of the next token given the query, a retrieved document, the tokens already produced and the system prompt. Because the query, context and system prompt together determine the model’s input, changing any one of them can change the output, which is why the others must be held fixed if a change in behaviour is to be attributed to the system prompt alone.

Decoding and the Temperature Parameter

Generation is not deterministic by default. At each step the model produces a score u_v for every token v in the vocabulary $\mathcal{V}$, and these scores are turned into a probability distribution by the softmax function with a temperature parameter $T > 0$:

$$p(v) = \frac{\exp(u_v/T)}{\sum_{v' \in \mathcal{V}} \exp(u_{v'}/T)}. \quad (2.1)$$

The temperature rescales the scores before they are normalised, and it controls how much probability mass is left for tokens the model considers less likely (Holtzman et al., 2020). As T approaches zero the distribution concentrates on the highest-scoring token, and decoding becomes effectively deterministic: the same input produces the same output. As T grows the distribution flattens, lower-scoring tokens become reachable, and the output becomes more varied across runs.

For open-ended writing this variety is useful. For a RAG system it is mostly a liability. A token that the model scores as unlikely given the retrieved context is, by construction, one that the evidence does not support well, so raising the temperature raises the chance of producing a statement the retrieved documents do not contain. High temperature also makes the system harder to evaluate, since the same query and the same evidence no longer yield the same answer. RAG systems intended to be faithful to their sources are therefore usually run at a low temperature, trading variety for grounding and reproducibility. This thesis holds T fixed and low across all experimental conditions; the residual randomness that remains at a fixed temperature is the reason each condition is repeated several times in Chapter 5.

2.2.4 System Prompt

The system prompt S specifies the instructions, constraints and output requirements the generator should follow, independent of the specific query or retrieved context. Unlike the query, which changes with every request, and the retrieved context, which changes with the query and the knowledge base, the system prompt is usually fixed across many requests.

One terminological point has to be settled here, because Chapter 3 reviews a literature that does not settle it. In this thesis *system prompt* denotes S alone: the standing instruction supplied in the system role by whoever deploys the system. Much published work on prompt sensitivity uses *prompt* more broadly, for the entire input the model receives, or more narrowly, for the task instruction and any worked examples accompanying the question in the user turn. Under either reading the text being varied is what this thesis calls the query q , or the query together with its framing, and not S . The difference is a difference in who writes the text and how often it changes: q arrives with each request from whoever is using the system, whereas S is written once and applies to every request thereafter. A finding about the one does not automatically carry over to the other, and Chapter 3 keeps the two bodies of work apart for that reason.

Its role in the architecture is easiest to see by asking what happens without it. Retrieval places relevant evidence in the context window, but placing evidence there does not oblige the model to use it. Section 2.2.3 described the parameters θ as a store of information absorbed during training, and

that store remains available at inference time. A model given a retrieved passage and a question it believes it already knows the answer to may answer from parameters rather than from the passage, producing an assured and fluent response that the retrieved documents do not support. This failure is not signalled to the user, since fluency and confidence are unaffected by whether the underlying claim was grounded; Niu et al. (2024) document that unsupported statements continue to appear even when relevant evidence has been retrieved and supplied. The system prompt is the last point in the pipeline at which this can be constrained. It is the only component that can state, before generation begins, that the retrieved context is the sole admissible source of fact.

The system prompt accordingly has two distinct functions. The first is to define the operational scope of the system: the role it occupies, the audience it addresses, the register it writes in, and the format its output takes. The second is to install hard constraints on that behaviour, of which the grounding constraint is the most consequential. A grounding constraint restricts the model to the retrieved context and specifies what to do when that context is inadequate, which converts an unanswerable query from a case the model improvises around into a case with a defined response. The two functions differ in kind: scope shapes how an answer is written, whereas a hard constraint determines whether an answer may be given at all. Listing 2.1 illustrates both in a minimal example.

```
You are a customer service assistant for a parcel delivery company.

GROUNDING RULE
Answer only from the context provided below. Do not use information
from your training data, even where you are confident that it is
correct.

INSUFFICIENT EVIDENCE
If the context does not contain the answer, reply with exactly:
The provided documents do not contain this information.

FORMAT
Answer in at most three sentences. Cite the identifier of every
document you use, in square brackets.
```

Listing 2.1: A minimal system prompt combining an operational scope with two hard constraints.

The first line fixes the operational scope. The two blocks that follow are hard constraints: one restricts the admissible source of fact, the other defines the response for the case in which no admissible source exists. The final block constrains form rather than content, and belongs to the first function. Each of these instructions is stated in a way that can be checked against an output, which is the property Chapter 4 requires of the requirement set Σ .

Whether an instruction of this kind is in fact obeyed is a separate question from whether it has been written. Because the system prompt is expressed in natural language rather than as a specification the generator is bound to satisfy, its effect faces the same instruction-following and prompt-sensitivity issues as any other natural-language instruction to a language model, discussed further in Chapter 3. A grounding constraint that is followed for one phrasing and disregarded for another provides weaker protection than its wording suggests. This is why the system prompt, formalised in Chapter 4 as a set of requirements together with a wording, is the object of experimental control in this thesis.

2.3 Retrieval-Augmented Generation and Fine-Tuning

Fine-tuning is the other established way of adapting a general-purpose language model to a specific application, and it is useful to state how it differs from retrieval before the use cases in Section 2.4 are discussed. Fine-tuning continues training on domain-specific data and changes the model parameters θ , so what is learned is absorbed into the parametric memory described in Section 2.2.3. RAG leaves θ untouched and changes what is placed in the context window at inference time. The two are therefore not competing answers to one question. They act on different parts of the system, and the practical decision is which part a given requirement belongs to: the knowledge the system reasons over, or the way the system reasons and writes (Ovadia et al., 2024).

For knowledge that is new, changing, or proprietary, retrieval is the better fit, for three reasons. The first is cost. Fine-tuning requires a training run over the model’s parameters whenever the underlying information changes, whereas adding a document to a knowledge base requires only that the new chunks be embedded and indexed, which is a fraction of the computation and can be done continuously rather than in scheduled retraining cycles (Cheerla, 2025). The second is data freshness and revocability. A document can be added, corrected, or deleted and takes effect at the next query, while a fact absorbed into θ cannot be selectively removed without retraining; Wang et al. (2024) survey the difficulty of locating and editing individual facts inside model parameters. This matters wherever information is corrected, retracted, or must be deleted on request, and it is one reason retrieval is preferred in regulated settings, where sensitive data can remain in a controlled external store instead of being absorbed into the parameters (He et al., 2025). The third is auditability. Because a RAG answer is produced from a specific set of retrieved chunks, each claim in the output can in principle be checked against the passage that supports it, and unsupported claims can be identified at the level of the individual statement (Niu et al., 2024). A fine-tuned model has no comparable trace: the information is distributed across its parameters, and the model cannot indicate which training example a given assertion came from. Retrieval does not eliminate unsupported claims, as Section 2.4 discusses, but it does make them detectable.

Fine-tuning remains the correct choice when the requirement concerns behaviour rather than facts (Zhou et al., 2023a). Three cases are typical. The first is specialised vocabulary: in a domain whose terminology, abbreviations, or notation are poorly represented in the training corpus, a model may misread the query or the retrieved evidence itself, and no amount of retrieval repairs a failure that occurs during interpretation. The second is strict output structure. A system that must emit a fixed JSON schema, a proprietary markup or a domain-specific programming language applies that constraint to every request, and a constraint that holds for every request is more reliably and more cheaply placed in the parameters than restated in every prompt. The third is register and tone, where a consistent style is wanted across many interactions rather than described anew each time. In each case what is being adjusted is how the model responds, not what it knows.

Framed this way, the two techniques are complementary rather than exclusive, and production systems often combine them: a model fine-tuned for the vocabulary and output format of its domain, retrieving the facts it reasons over from an external knowledge base. The separation matters for this thesis because behaviour can be controlled through either channel. Fine-tuning writes the desired behaviour into θ ; the system prompt requests it in natural language at inference time. The second channel is far cheaper and can be changed at any moment, which is why it is the one most systems rely on, but it is also the one whose effect is hardest to predict, since a natural-language instruction is not a specification with guaranteed semantics. This thesis holds θ fixed and studies that second channel.

2.4 Use-Case of RAG

RAG systems are typically used where the needed knowledge is too large, too specific, or changes too often to be captured reliably by a model’s parametric knowledge alone. Concrete deployments illustrate this across several domains: question-answering over enterprise and internal document collections (Cheerla, 2025), customer support systems that must answer from current product or policy documentation, coding assistants that retrieve from a specific codebase instead of relying on general programming knowledge, legal question-answering over case law and contract repositories (Pipitone and Alami, 2024), and clinical decision support that grounds its answers in current medical literature and institutional protocols (He et al., 2025). In each of these cases, RAG is usually preferred over two alternatives. The first is fine-tuning the generator on the target document collection, which Section 2.3 discusses: the collections above change continuously, and retraining on each change is neither economical nor traceable. The second is extending the model’s context window to include the whole document collection, which avoids retraining but does not scale to large collections and does not, by itself, make sure the model actually attends to the passages that are relevant to the query.

Despite these advantages, RAG does not remove the failure modes of purely parametric generation; it shifts them, and in some cases only partly fixes them. Chen et al. (2024a) show that current language models tolerate a moderate amount of irrelevant retrieved content reasonably well, but still struggle to reject a query when none of the retrieved evidence is enough to answer it, to combine evidence spread across multiple retrieved chunks, and to stay robust when retrieved evidence is outdated or contradictory. Niu et al. (2024) further show that language models keep producing hallucinated content even when relevant evidence has been retrieved and given to them in context. This means that grounding a model in retrieved evidence does not, by itself, guarantee that its output stays faithful to that evidence. These failure modes are why the literature review in Chapter 3 and the experimental design in Chapters 4 and 5 build the query set around directly answerable and multi-hop cases and vary the quality of the retrieved evidence separately, so the system prompt’s effect on exactly these failure modes can be measured.

2.5 Advanced RAG Architectures

The architecture described so far retrieves once per query and generates once from what was retrieved. Recent work relaxes both assumptions, and the resulting variants can be grouped by which stage of the pipeline they modify. The following summary is included for completeness; none of these variants is used in this thesis, which studies the standard architecture.

Conversational RAG. Retaining prior turns of a dialogue as state, so that a follow-up question can be resolved against what was already asked and answered. A query such as “and the second one?” is meaningless to a retriever on its own, so the conversation history is used either to rewrite it into a standalone query or, as Liu et al. (2024b) show, to train a retriever directly on multi-turn data, which reaches comparable accuracy at lower cost than rewriting.

Pre-retrieval optimisation. Rewriting or expanding the query before it reaches the retriever. Hypothetical Document Embeddings (HyDE) generate a hypothetical answer to the query and embed that instead of the query itself, on the argument that an answer lies closer in embedding space to the passage containing it than a question does (Gao et al., 2023a).

Routing and adaptive retrieval. Deciding where and how much to retrieve. Where several collections or indexes are available, a routing step can select among them according to what the query is about, rather than searching one combined collection for everything. Adaptive-RAG goes further and varies the depth of retrieval with the difficulty of the query, answering simple questions directly and reserving multi-step retrieval for those that need it (Jeong et al., 2024).

Self-correction and reflection. Evaluating retrieved evidence before or during generation. Corrective RAG (CRAG) grades each retrieved document, decomposes the ones it keeps into smaller knowledge strips and discards the strips that are not relevant, falling back to a wider search when retrieval fails (Yan et al., 2024). Self-RAG instead trains the model to decide for itself when to retrieve and to critique its own draft output, so that retrieval and generation interleave rather than run once each (Asai et al., 2024).

Agentic RAG. Treating retrieval as one action among several available to an agent. A meta-agent delegates to document-specific or tool-specific agents and combines their results, which turns a fixed pipeline into a workflow whose steps are chosen at run time (Singh et al., 2025).

Other work extends the architecture in directions that are orthogonal to this routing taxonomy: knowledge graphs structure the retrieved evidence rather than treating it as a flat set of passages (Edge et al., 2024), and multimodal retrieval covers images, audio and video alongside text (Abootorabi et al., 2025). A recent systematic review places these directions within a single trajectory away from single-shot, text-only retrieval (Oche et al., 2025). Understanding the standard architecture formalised in this thesis remains a prerequisite for all of them: each varies the retrieval function R , the generation function G , or the number of times they are applied, but none changes the underlying distribution $p_{\text{RAG}}(y \mid q, D, S)$ on which they still depend.

This thesis implements the standard architecture and none of the variants above. The reason is methodological. The research objective is to estimate the causal effect of the system prompt on the output, which Chapter 4 formalises as the intervention $\text{do}(S = S_j)$ and which is identifiable only as long as the system prompt reaches the answer through a single path. Each of the variants breaks that condition. Under Self-RAG the model decides for itself when to retrieve (Asai et al., 2024), so the system prompt influences which documents are fetched as well as how they are used. Under routing, corrective and agentic designs the prompt shapes the decisions of the router, the retrieval evaluator or the meta-agent, so the retrieved context C ceases to be determined by the query and the corpus alone. Retrieval would then have to be written $p_{\eta}(z \mid q, D, S)$ rather than $p_{\eta}(z \mid q, D)$, the arrow from S into the retriever that Figure 4.1 deliberately omits would be present, and a measured difference between two prompt conditions could be attributed either to the prompt’s effect on generation or to its effect on what was retrieved, with no way to separate the two. The routing policy would operate as a confounding variable, and the isolation of effects on which the experimental design rests would no longer hold.

The standard architecture is therefore the controlled baseline rather than a simplification adopted for convenience. It is the configuration in which the system prompt enters the pipeline at exactly one point, which is the property that makes the intervention identifiable. The direction of inference matters as well: because the variants above add routing, reflection or delegation around the retrieval and generation functions instead of replacing them, an effect established at this level bounds what can be expected once those mechanisms are introduced, whereas an effect measured inside an agentic pipeline could not be decomposed back into a prompt component and a routing component. It is also the configuration most widely deployed in practice, as Section 2.4 describes, so the result is not

restricted to a hypothetical case. Establishing the causal mechanics of prompt interventions at this level is a prerequisite for studying them in any richer pipeline, and extending the design to one of the variants above is left to future work.

3 Literature Review

Chapter 2 introduced the components of a RAG system. This chapter reviews three areas of literature relevant to studying how the system prompt affects such a system: Section 3.1 covers prompt sensitivity and instruction-following in language models in general, Section 3.2 covers retrieval-augmented generation specifically, its architecture, evaluation, and known failure modes, and Section 3.3 covers adversarial and competing instructions delivered through retrieved content. Section 3.4 brings all three together into the specific research gap this thesis addresses.

3.1 Prompt Sensitivity and Instruction-Following in Language Models

The first branch of literature studies language models with no retrieval component at all. It divides on a distinction that decides what can be inferred from it. One group manipulates the system prompt in the sense used here, a standing instruction supplied in the system role and held apart from the user’s question. A larger group manipulates what this thesis calls the query, or the task instruction and worked examples that accompany it in the user turn, and calls that object the prompt. Both groups report that behaviour moves under changes leaving the propositional content of the instruction intact, but only the first speaks directly to the variable manipulated here.

Three studies vary the system prompt itself. Cheng and Mastropaolo (2026) vary the system prompt given to instruction-tuned models for code generation, crossing prompt specificity and prompting strategy with model, programming language and decoding settings, and report that the system prompt moves the quality of generated code independently of the task description. Theirs is the closest published design to the one used here, and the present experiment carries it from code generation into a retrieval-augmented setting, where the model is also given evidence to work from. Mu et al. (2025) examine how reliably models honour a system prompt when the user turn contains instructions conflicting with it, which establishes the system prompt as a control whose authority is contingent rather than guaranteed. Zheng et al. (2024) assign personas through the system prompt across a large set of roles and find that doing so does not reliably improve accuracy over a system prompt carrying no persona at all. That result is worth stating separately, because it distinguishes changing the system prompt from improving the system: an intervention on S can alter behaviour without moving any outcome measure in the intended direction.

A larger group varies the user-side prompt. Lu et al. (2024) compare prompting techniques in in-context learning and measure sensitivity through gradient-based saliency scores, reporting that sensitivity correlates negatively with accuracy and can serve as an unsupervised proxy for it. Zhuo et al. (2024) propose ProSA, which scores instance-level variation in the task instruction against decoding confidence, and report that larger models are more robust to such variation and that few-shot examples reduce it. Salinas and Morstatter (2024) perturb task prompts across text classification tasks, adding a trailing space, requesting the output in XML, or prefixing a jailbreak, and find that perturbations of this size change the labels the model assigns. Sclar et al. (2024) vary only the formatting of few-shot task templates, the separators, casing and spacing carrying no

propositional content, and report differences of up to 76 accuracy points between formats a reader would judge equivalent. Hua et al. (2025) run twelve task templates across several benchmarks and argue that much of the measured sensitivity is produced by the evaluation method, log-likelihood scoring and rigid answer matching, rather than by the model, since the variance falls once an LLM judge is used instead. Qin et al. (2026) decompose a model’s output score into interactions between input variables and observe that small changes to the prompt destabilise those interactions even where the final output is unchanged.

Two further studies sit outside both groups. Ruiz and Cardona-Betancourt (2026) state a rule in the instruction given to a model and then shift the contingencies that rule was written for, finding that models keep following the stated rule after it has stopped paying, a pattern resembling rule-governed behaviour in humans. The channel carrying the rule is not the variable of interest there, but the finding bears on how a stated requirement competes with what the evidence in front of the model actually supports. Ying et al. (2024) study conflict between a model’s parametric memory and the evidence supplied in its context, classifying models as intuitive or dependent by which source they favour. That conflict runs between memory and context, not between a system prompt and a user instruction, so the study is evidence about the evidence axis of this thesis rather than about its prompt axis.

Two conclusions follow, and they carry unequal weight. The first is that the system prompt is a behavioural control variable whose effect can be isolated and measured given a properly controlled setup, which the three system-prompt studies support directly (Cheng and Mastropaolo, 2026; Mu et al., 2025; Zheng et al., 2024). The second is that instructions are sensitive to changes preserving their meaning, whether lexical (Lu et al., 2024; Zhuo et al., 2024; Salinas and Morstatter, 2024), purely formatting-level (Sclar et al., 2024), or not visible in the output at all (Qin et al., 2026), and that the apparent size of such an effect depends on how it is measured (Hua et al., 2025). This second conclusion is established for the user-side prompt, and its transfer to the system prompt is an assumption rather than a result. The assumption is a reasonable one, since both texts reach the same decoder as tokens in the same context window, but it is untested, and instruction tuning gives a reason it might fail: models are trained to treat the system role as carrying more authority than the user role, which could make S either more stable under rewording or less. Which of these holds is part of what this thesis sets out to determine rather than a premise it inherits. None of these studies, in either group, works inside a retrieval-augmented setting, where behaviour also depends on evidence that may be noisy, conflicting, or absent.

3.2 Retrieval-Augmented Generation: Architecture, Evaluation, and Failure Modes

The second branch narrows to retrieval-augmented generation itself, and to the failure modes it shows even when no one is manipulating the system prompt.

Lewis et al. (2020) introduce Retrieval-Augmented Generation, combining a pretrained neural retriever with a pretrained sequence-to-sequence generator, and show that conditioning generation on retrieved evidence outperforms purely parametric models on knowledge-intensive tasks; their probabilistic formulation, in which retrieval is a distribution over documents and the retrieved document is marginalized out, is the direct basis for the model formalised in Chapter 4 of this thesis. Gao et al. (2023c) survey the development of RAG systems through Naive, Advanced and Modular paradigms, organising the field around a tripartite retrieval-generation-augmentation structure; their taxonomy supports treating retrieval and generation as separable functions whose

interaction with the system prompt can be studied independently, as this thesis does. Two later developments extend this taxonomy in directions this thesis deliberately does not follow. Edge et al. (2024) restructure the retrieval side into a knowledge graph, grouped into thematic communities and pre-summarised by an LLM, so that a query is answered by traversing entities and community summaries rather than by a flat nearest-neighbor lookup over D . Singh et al. (2025) and Jin et al. (2025) restructure the generation side into an agent loop, in which the model itself decides when and what to retrieve, calling R repeatedly and adaptively rather than once per query; Jin et al. (2025) train this behaviour directly with reinforcement learning and report large gains over standard RAG baselines. Both directions relax exactly the components Chapter 4 holds fixed, R in the first case and the single-call structure of G in the second. For that reason this thesis retains Gao et al.’s simplest, Naive-RAG form: any change in the outcomes measured here can then be attributed to the system prompt, rather than to the added architectural complexity of a graph-structured retriever or an agentic generation loop.

Es et al. (2023) propose Ragas, a reference-free evaluation framework for RAG pipelines built around three metrics, faithfulness, answer relevance and context relevance, that require no ground-truth human annotation; two of those three, faithfulness and answer relevance, are adopted directly as measures in this thesis, while the third, context relevance, is a retrieval-side measure and is excluded for the reasons given in Section 5.4; their framework also motivates evaluating the effect of the system prompt independently of retrieval quality. Chen et al. (2024a) construct the Retrieval-Augmented Generation Benchmark (RGB) and show that current language models tolerate a moderate amount of retrieval noise but struggle with negative rejection, information integration across multiple retrieved passages, and robustness to counterfactual or contradictory evidence; their four testbeds map onto the experimental design of this thesis, the first two as query categories and the remaining two as levels of retrieved-evidence quality. Niu et al. (2024) introduce RAGTruth, a manually annotated corpus of word-level hallucinations produced by RAG systems, and benchmark hallucination-detection methods across multiple language models; their finding that hallucination persists even when relevant evidence has been retrieved motivates measuring faithfulness as an outcome distinct from retrieval quality, as this thesis does. Yu et al. (2024) draw this body of work together into a survey of RAG evaluation, separating the targets that assess the retrieval stage from those that assess generation and naming relevance, faithfulness and correctness as the three that belong to the latter; the measures adopted in Section 5.4 are taken from that account, and the separation it draws is the one on which this thesis’s restriction to generation-stage measurement rests.

Together, these studies establish the architecture, evaluation vocabulary, and known failure modes of RAG systems specifically. They show that a RAG system’s output quality depends on both retrieval quality and how faithfully the generator uses the retrieved evidence (Es et al., 2023; Niu et al., 2024), and that current systems already struggle with certain evidence conditions, such as conflicting or insufficient retrieved evidence, even without any prompt manipulation (Chen et al., 2024a). None of the studies in this branch, however, treat the system prompt as a controlled experimental variable; it is either fixed across conditions or not specified at all.

3.3 Adversarial and Competing Instructions in Retrieved Content

A third, closely related line of work studies what happens when instructions arrive not from the system prompt but from the retrieved content itself, competing with the system prompt for control of the model’s behaviour. Where the first branch shows that a model’s own system prompt can lose

authority to instructions arriving in the user turn (Mu et al., 2025), Greshake et al. (2023) show that an entirely different source of instruction, text hidden inside a document the model is told to process, can override the system prompt altogether. This is the adversarial extreme of the same instruction-competition problem, except the competing instruction arrives through the retrieved context C rather than through S , a source this thesis deliberately holds fixed rather than varies. Zhang et al. (2025) extend this observation into a RAG-specific benchmark, testing 13 poisoning attack methods and 7 defenses across 5 base question-answering datasets and 10 expanded variants; their findings sit alongside the observations of Chen et al. (2024a) and Niu et al. (2024) from the second branch that retrieved evidence can already be noisy, incomplete, or contradictory, and show that it can also be made deliberately adversarial.

Both studies, however, treat instruction competition exclusively as a security problem: an adversary deliberately crafts malicious content, and the goal is to detect or defend against it. Neither examines benign, non-adversarial linguistic variation of the system prompt itself, of the kind the prompt-sensitivity literature reviewed above studies outside of RAG, and of the kind this thesis’s conditions ($S_{b,0}$ through $S_{b,4}$, for each baseline b) are built to isolate inside it. This distinction matters: this thesis is not concerned with whether a RAG system can be attacked, but with whether ordinary, well-intentioned rewordings of its own instructions change its behaviour at all.

3.4 Synthesis and Research Gap

The first branch shows that the system prompt is a controllable way of shaping language model behaviour, and, as a separate finding, that instructions are sensitive to rewording that leaves their meaning intact. It establishes the first for the system prompt and the second mostly for the user-side prompt, and it studies both outside any retrieval-augmented setting. The second branch establishes the architecture, evaluation vocabulary, and known failure modes of RAG systems, including known problems with evidence integration, rejection, and faithfulness, but does not treat the system prompt as a controlled experimental variable. The third branch shows that instructions competing with the system prompt is a real and actively studied phenomenon in RAG systems, but only as a deliberate security threat delivered through retrieved content, never as benign, controlled linguistic variation of the system prompt itself. The overlap between all three branches is still largely unexamined. Ying et al. (2024) comes nearest to it, but the conflict studied there runs between a model’s parametric memory and its retrieved context rather than between two wordings of one instruction, and neither retrieval nor the underlying model is held fixed as an explicit experimental control.

This thesis addresses that gap directly. It treats the system prompt as a controlled experimental variable inside a RAG pipeline, holding the query, knowledge base, retrieval mechanism, and underlying language model fixed, so that any change in system behaviour and outcomes can be traced to the prompt itself. Four of the five conditions vary only the wording of the instructions while leaving what they require untouched, which isolates the effect of phrasing from the effect of content; the fifth alters the requirements themselves by weakening their modal force, and is kept separate in the analysis precisely so that the two kinds of change are never conflated.

The methodological form this takes is not arbitrary. The experimental design of this thesis is grounded in the causal inference framework for natural language processing set out by Feder et al. (2022), who consolidate work scattered across several fields into a single account of what it means to estimate a causal effect when the variables of interest are texts. Their organising distinction is between the three roles a text can occupy in a causal question: it may be the outcome, the confounder, or the treatment. The study undertaken here belongs to the third of these. The system

prompt is a text assigned by the experimenter rather than observed in the wild, and the quantity of interest is the effect of assigning one such text rather than another. Two of their arguments bear directly on the design that follows. The first is that a causal question about language demands that the intervention be formalised before it can be estimated, since natural language admits no self-evident notion of what it would mean to change one thing and hold the rest constant. The second is their observation that where domain knowledge does not supply a small set of admissible texts, controllable generation offers a way to assign texts that differ in only one latent aspect, changing the style of a text while holding its content fixed. That prescription describes the construction of the prompt conditions used here, in which the set of requirements a prompt states is held fixed while the wording expressing them is varied, so that the assigned texts differ along a single axis. Applying this philosophy to a retrieval-augmented system adds a demand their single-model examples do not face: the treatment must be shown to reach the outcome along one path only, which is why the retriever and the evidence it returns are held fixed throughout. Chapter 4 builds the formal model that makes this control precise, describing the RAG system as a retrieval function combined with a generation function, with the system prompt as an explicit parameter of the generation function, and stating the assignment of a prompt condition as an intervention on that model.

4 Formal Model of a RAG System

Chapter 3 identified a gap: prompt sensitivity is studied outside retrieval, and RAG evaluation does not treat the system prompt as a controlled variable. Closing it requires a model of the system precise enough to isolate the system prompt. This chapter builds that model on the probabilistic formulation of retrieval-augmented generation given by Lewis et al. (2020), and extends it by making the system prompt an explicit variable of the generation process.

4.1 Modelling Objective

The formalisation serves two purposes. It states the probabilistic relationship between the user query, the retrieved documents and the generated output, and it provides the basis for treating the system prompt as an experimental intervention whose effect can be measured while the query, the knowledge base, the retriever and the language model are held constant.

The model can be read as a chain,

$$(q, D) \longrightarrow p_\eta(z \mid q, D) \longrightarrow p_\theta(y_i \mid q, z, y_{1:i-1}, S) \longrightarrow p_{\text{RAG}}(y \mid q, D, S),$$

in which the system prompt S enters at the generation stage only. The research question of this thesis is then a question about that final distribution: what changes in $p_{\text{RAG}}(y \mid q, D, S)$ when S is changed?

The symbols used below are those listed in the Notation section of Chapter 1. The one departure from Lewis et al. (2020) is that they write the input sequence as x , whereas this thesis writes it as q , since the input is always a user query; their notation is otherwise kept unchanged.

4.2 Probabilistic Retrieval

Retrieval is represented as a probability distribution over documents rather than as a single choice:

$$p_\eta(z \mid q, D). \tag{4.1}$$

Equation (4.1) reads as the probability the retriever assigns to document z , given the query q and the knowledge base D . Writing retrieval this way means the retriever never has to commit to one document being the correct one. It spreads probability over the candidates, and documents it considers more relevant receive more of it.

A knowledge base may contain a very large number of documents, and evaluating all of them is not practical. The formulation therefore keeps only the K documents with the highest retrieval probability,

$$\mathcal{Z}_K(q, D) = \underset{z \in D}{\text{TopK}} \ p_\eta(z \mid q, D), \tag{4.2}$$

and the retrieved context handed to the generator is the resulting collection,

$$C = \{z_1, \dots, z_K\}, \quad z_k \in \mathcal{Z}_K(q, D). \quad (4.3)$$

The distinction between z and C matters later: the first is a candidate the model sums over, the second the concrete collection the running system retrieves and logs.

4.3 Prompt-Conditioned Generation

Lewis et al. (2020) write the generator at the level of a single token as $p_\theta(y_i \mid q, z, y_{1:i-1})$: the probability of the next token given the query, one retrieved document, and the tokens already produced. This thesis adds the system prompt as a further conditioning variable:

$$p_\theta(y_i \mid q, z, y_{1:i-1}, S). \quad (4.4)$$

Equation (4.4) is the point at which this thesis departs from the original formulation, and the departure is deliberately small. The generator is the same model with the same parameters θ ; what changes is that the system prompt is now written into the conditioning information rather than left implicit.

Because generation is autoregressive, the probability of a complete output sequence follows from the token probabilities by multiplication:

$$p_\theta(y \mid q, z, S) = \prod_{i=1}^L p_\theta(y_i \mid q, z, y_{1:i-1}, S). \quad (4.5)$$

4.4 Marginalizing over Retrieved Documents

Equation (4.5) gives the probability of an output under one particular document z . Since the system does not know which retrieved document is the one that supports the answer, the model averages over the candidates, weighting each by its retrieval probability. Lewis et al. (2020) give two ways of doing this, which differ only in where the sum is placed.

4.4.1 RAG-Sequence

RAG-Sequence assumes that a single retrieved document is responsible for the whole answer. The probability of an output is the weighted average, over documents, of the probability that document would produce that entire sequence:

$$p_{\text{RAG-Sequence}}(y \mid q, D, S) \approx \sum_{z \in \mathcal{Z}_K(q, D)} p_\eta(z \mid q, D) \prod_{i=1}^L p_\theta(y_i \mid q, z, y_{1:i-1}, S). \quad (4.6)$$

The sum stands outside the product, which is what encodes the assumption: one document is fixed, the entire sequence is generated from it, and only then is the next document considered.

4.4.2 RAG-Token

RAG-Token makes the weaker assumption that different parts of an answer may come from different documents. The average over documents is taken separately at every token:

$$p_{\text{RAG-Token}}(y \mid q, D, S) \approx \prod_{i=1}^L \sum_{z \in \mathcal{Z}_K(q, D)} p_\eta(z \mid q, D) p_\theta(y_i \mid q, z, y_{1:i-1}, S). \quad (4.7)$$

The whole difference between the two is the order of the two operators: $\sum_z \prod_i$ against $\prod_i \sum_z$. RAG-Sequence commits to one document for the length of the answer; RAG-Token lets one document supply the first half of a sentence and another the second, which is the more flexible assumption and the one better suited to answers that combine several sources.

Both are written here as $p_{\text{RAG}}(y \mid q, D, S)$ where the distinction does not matter, and it does not matter for the experiment: the system prompt S appears in exactly the same place in both, inside the generator and nowhere else.

$$p_{\text{RAG}}(y \mid q, D, S). \quad (4.8)$$

4.5 The System Prompt: Requirements and Wording

The system prompt enters Equation (4.4) as a single symbol, but it is a text with internal structure, and the experiment varies that structure deliberately. A prompt states a set of requirements and states them in particular words. Writing the two separately gives

$$S = \rho(\Sigma, \varphi), \quad \Sigma = \{\sigma_1, \dots, \sigma_m\}, \quad (4.9)$$

which separates what a prompt asks for from the words it asks in. Two prompts built from the same Σ ask for the same thing; two built from the same φ say things the same way.

The conditions cross two factors. The first is the requirement set. Three are used, each taken from a prompt published with a retrieval-augmented system and reproduced in Section 5.5. Writing Σ_A , Σ_B and Σ_C for the three: Σ_A is the response-generation prompt of Friel et al. (2024), which states a single requirement; Σ_B is the VANILLA instruction of Gao et al. (2023b), which states about ten clauses falling into four independently checkable families; and Σ_C is the instruction of Mao et al. (2024), which its authors describe as three parts. The sets differ in m by an order of magnitude between Σ_A and Σ_B , with Σ_C lying between them.

The second factor is the wording. Four variants are written for each requirement set, holding Σ fixed and changing only φ , and a fifth changes the requirement set itself. The axes along which the wording is varied are not devised here. They follow Wahle et al. (2024), who vary prompts by linguistically defined paraphrase type rather than by ad hoc rewriting and group those types into families of which morphology, syntax and lexicon are the ones that apply to a standing instruction; Zhan et al. (2024), who isolate the lexical axis by substituting single semantically similar words and report that models are over-sensitive to variations imperceptible to a reader; and Kostić et al. (2026), who separate a lexical pipeline built on synonym substitution from a syntactic one built on dependency parsing and find the two to behave differently, lexical perturbation degrading performance consistently while syntactic perturbation does not. This thesis follows their approach of varying a prompt along linguistically defined, meaning-preserving axes one at a time, and applies it to the standing system prompt of a retrieval-augmented system, which is what its research question requires and what none of the three addresses. Writing $S_{b,j}$ for the j -th wording of baseline b :

$S_{b,0} = \rho(\Sigma_b, \varphi_0)$, **canonical**. The published form, unchanged. The reference the other four are measured against.

$S_{b,1} = \rho(\Sigma_b, \varphi_1)$, **lexical**. Synonym substitution with clause order and voice held fixed.

$S_{b,2} = \rho(\Sigma_b, \varphi_2)$, **syntactic**. Clause reordering and active to passive, with the vocabulary held as far as English allows.

$S_{b,3} = \rho(\Sigma_b, \varphi_3)$, **format**. The same wording laid out as a list rather than as prose.

$S_{b,4} = \rho(\Sigma'_b, \varphi_0)$, **deontic**. Canonical wording with every obligation weakened to a request.

Fifteen conditions follow. Each axis changes one thing and holds the rest still, which is what makes a difference between two conditions attributable to that axis, and which is easy to break by accident. The syntactic variant carries obligation uniformly through the construction *is to be*, so that a change of voice does not also change modal force; the format variant writes its items as bare noun phrases for the same reason. Modal force belongs to the deontic condition alone.

Baseline *A* states the fewest requirements and is short enough to give in full. The five conditions read:

A0 Use the following pieces of context to answer the question.

A1 Employ the following excerpts of material to address the query.

A2 The question is to be answered using the context pieces below.

A3 CONTEXT: the following pieces
TASK: use them to answer the question

A4 Please try to answer the question using the context pieces below.

Listing 4.1: The five conditions of baseline *A*, whose canonical form is the prompt of Friel et al. (2024).

The variants for baselines *B* and *C* are written on the same principle and are longer in proportion to their requirement sets. Within each baseline the five conditions are held to within roughly a tenth of the canonical form in length, so that a difference between two of them cannot be read as an effect of how much text the prompt contains rather than of the axis being varied. The deontic condition is the one that resists this: weakening an obligation to a request adds words, and forcing it back to the canonical length would mean cutting a requirement, which would change Σ rather than φ . It is therefore allowed to run slightly long, and the word counts of all fifteen conditions are recorded so that length can be entered as a covariate if the results call for it.

Crossing the wording axes with three requirement sets rather than one is what lets the central question be asked properly. If the system responded only to what a prompt requires and not to how it is written, $S_{b,0}$ through $S_{b,3}$ would produce the same output distribution within every baseline, since they differ in φ alone. Whether they do is the empirical question, and the format condition in particular follows Sclar et al. (2024), who report large performance differences from formatting changes of this kind. They apply them to few-shot task templates rather than to a system prompt, so carrying the manipulation across to *S* is itself part of what is being tested here. Using three baselines also answers an objection a single one cannot. A prompt stating one requirement leaves the deontic condition almost nothing to weaken, while a prompt stating ten risks confounding the wording manipulation with the sheer length of the text. Observing the same wording effect across requirement sets that differ by an order of magnitude in *m* separates the effect of wording from the effect of density; observing it in only one leaves the two joined.

Every contrast is taken within a baseline. Comparing $S_{B,1}$ against $S_{A,0}$ would confound a change of wording with a change of what was asked for, so each variant is measured against the canonical form of its own requirement set.

4.6 The System Prompt as an Intervention

The prompt appears in the generator, in Equation (4.4), and not in the retriever, in Equation (4.1). The retrieval term is therefore written without S among its arguments, which states that the system prompt is supplied to the generator but plays no part in choosing documents. In the notation of Pearl (2009), S has an effect on y and no effect on z . Figure 4.1 shows the resulting structure.

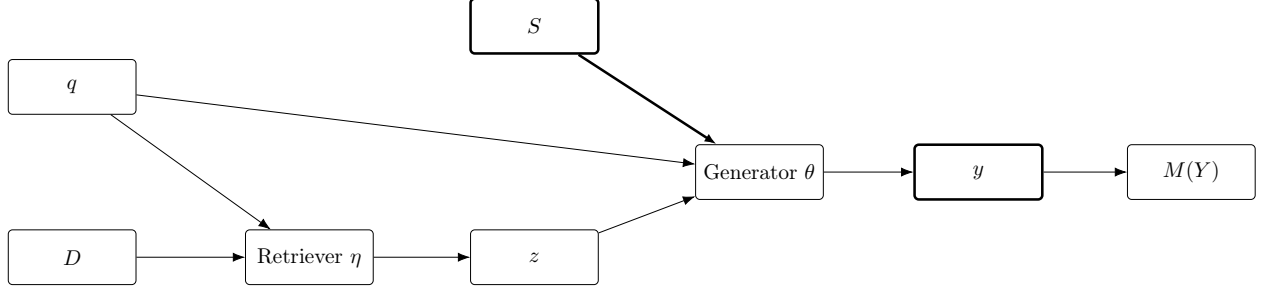


Figure 4.1: The system prompt S and the output y are drawn in bold. S enters the generator and has no arrow into the retriever, which is what allows a change in y to be attributed to a change in S .

For the effect of the prompt to be interpretable, the following are held constant across conditions. Each query is held fixed across the conditions it is run in, so that every comparison is made within a query rather than between queries, and the knowledge base is fixed at $D = D_0$. The retriever and its parameters are fixed at $\eta = \eta_0$, and the language model at $\theta = \theta_0$, so the model is neither retrained nor replaced between conditions. Retrieval is assumed prompt-independent,

$$p_\eta(z \mid q, D, S) = p_\eta(z \mid q, D), \quad (4.10)$$

which holds by construction for the single-shot architecture of Chapter 2 and is enforced in the experiment by recording the retrieved context once and reusing it. Generation, by contrast, is prompt-dependent: although θ stays fixed, the conditional distribution it defines may differ between S_0 and S_j . Finally, decoding is stochastic, so the same configuration can produce different outputs on different runs, which the term ε carries:

$$Y = G(q, z, S; \theta, \varepsilon), \quad (4.11)$$

which is why each condition is repeated several times in Chapter 5.

4.7 The Causal Effect

Setting the system prompt to a fixed value is an intervention, written $\text{do}(S = S_j)$ following Pearl (2009). The notation separates *setting* a variable from merely *observing* it: observing that two systems with different prompts give different answers leaves open that something else differed too, whereas setting the prompt while holding the rest fixed does not. For a query q under prompt condition S_j , the output of the system is the random variable

$$Y \sim p_{\text{RAG}}(y \mid q, D, S_j). \quad (4.12)$$

For one query the effect of the prompt is the difference in the expected value of the measured property $M(Y)$ between the two conditions. That difference is not, however, the quantity of interest. A prompt that improves the answer to one question and degrades the answer to the next has

established no effect at all, and a difference observed on a single input cannot be separated from the particular difficulty, phrasing or subject matter of that input. What the research question asks about is the behaviour of the system over the queries it will actually meet, so the effect has to be defined as an average over the distribution from which queries are drawn. Writing P_Q for that distribution over the query space $\mathcal{Q}$, the causal effect of prompt condition S_j relative to the baseline S_0 is

$$\tau_j = \mathbb{E}_{q \sim P_Q} \left[\mathbb{E}_{Y \sim p_{\text{RAG}}(\cdot|q,D,S_j)}[M(Y)] - \mathbb{E}_{Y \sim p_{\text{RAG}}(\cdot|q,D,S_0)}[M(Y)] \right], \quad (4.13)$$

which connects the probability model of Section 4.4 directly to the quantity the experiment estimates. Equation (4.13) is what Chapter 5 measures.

Equation (4.13) is an average treatment effect in the standard sense of that term: the query plays the role of the unit, the prompt condition the role of the treatment, and τ_j the mean over units of the difference between the two treatments. Two features of how it is written matter. The difference is taken inside the outer expectation rather than between two separately averaged quantities, which is what makes each query its own control and matches the paired design of Section 4.6; by linearity of expectation the two forms agree, but only the form given here records the pairing. And the expectations are nested rather than pooled: the inner expectation averages over the randomness of retrieval and decoding at a fixed query, while the outer averages over the queries themselves, so a large effect confined to a handful of inputs is not mistaken for a general one.

The distribution P_Q is not known and cannot be written down. Chapter 5 replaces it with the empirical distribution of a finite query set, which is the point at which the choice of data becomes a claim about generality rather than a matter of convenience, and the reason Section 4.8 draws the queries from established question-answer datasets rather than composing them for the experiment.

Two points about the interpretation. The intervention is on the conditioning variable S , not on the model parameters: θ remains θ_0 throughout, and the prompt changes what the generator is conditioned on rather than what it has learned. And because S is assigned by the experimenter rather than observed, no further assumption about hidden common causes is needed; what remains to be established is only that the conditions in Section 4.6 actually hold in the implementation.

Which property M should be is left open here. Section 4.9 describes what that property has to capture, and Section 5.4 fixes it.

4.8 Grounding the Knowledge Base in Established Datasets

The knowledge base has so far been written as a set $D = \{d_1, \dots, d_N\}$ of unanalysed objects, and the model says nothing about where those objects come from. Filling D is a methodological decision rather than a further step of formalisation, and it is settled here because the effect defined in Equation (4.13) is only as informative as the corpus and the queries it is estimated over.

Two routes are available. The first is to define the corpus theoretically: fix a set of facts about a small invented subject area, generate documents from those facts, and derive the query set from the resulting support structure. Its advantage is that every fact is known to the experimenter by construction. Its defect is that the data model and the hypothesis are then written by the same hand, so the difficulty of the queries, the density of distractors and the frequency of contradictions all become free parameters chosen in full knowledge of the effect they are meant to reveal. A result obtained on such a corpus is also incomparable with anything else in the field, because no other system has ever been run on it.

The second route, taken here, is to instantiate D and the query set from established question-answer datasets. Three properties recommend it. The questions are naturally occurring rather than authored for the experiment, so the query distribution is not fitted to the hypothesis it is used to test. The corpora are public, versioned and split in a documented way, so a third party can repeat the measurement without reconstructing a private data generator. And because published results for a wide range of retrieval and generation systems already exist on these datasets, the effect estimated in Chapter 5 can be read against a background of prior work instead of standing alone. Reproducibility and comparability are properties of the benchmark, not of the formalism, and no amount of additional notation supplies them.

Only MS MARCO is used. Its queries were issued to Bing by people looking for something rather than composed for a benchmark, and they are not confined to a subject area, so the collection spans a wide and uneven range of topics. That heterogeneity is the reason for the choice. An instruction that governs behaviour on one domain and fails on another has established little about the system prompt, and a corpus narrow enough to have been memorised would let a model score well without consulting the retrieved evidence. Drawing the whole query set from one heterogeneous collection also holds the annotation convention fixed, which a mixture of sources with different labelling rules would not.

The collection is built from queries sampled from Bing search logs, with candidate passages retrieved for each query and free-form answers written by human judges (Nguyen et al., 2016). Its passage-ranking split supplies the relevance judgements on which dense retrievers are commonly trained and evaluated, and that is the role it plays here. The dense retrieval architecture itself follows Karpukhin et al. (2020), whose dual encoder was shown to retrieve more accurately than a sparse lexical baseline on Natural Questions and four further open-domain benchmarks. For the present purpose MS MARCO governs the retrieval half of the pipeline: query-passage pairs whose relevance was judged externally rather than asserted by the experimenter, which is what a study of the system prompt requires, since the prompt must be shown to act on the generator and not on the evidence reaching it. Its annotations also mark the items for which no supplied passage answers the query, which is the material the abstention category of Section 5.2 is drawn from.

MS MARCO provides the queries q , the documents that populate D , and, through its annotations, both the reference answer a_q and the fact set Φ against which an output can be judged. The point of the empirical route is that Φ is fixed by the annotation and by the indexed passages, not by the experimenter.

What they do not provide is a definition of a good answer. An annotation records which answer is correct; it says nothing about whether an answer is grounded in the evidence actually retrieved, whether it addresses the question directly rather than evading it, or whether it obeys the requirements the system prompt states. Those properties are the object of this thesis, and defining them is the subject of the next section.

4.9 What Counts as a Good Answer

Nothing established so far says what would count as an improvement. The preceding sections describe what the system does and how the influence of the system prompt can be separated from everything else that shapes an output, but any comparison between prompt conditions presupposes a standard against which outputs are judged, and no such standard follows from the architecture itself. It has to be stated, and stating it belongs to the model rather than to the implementation: a measurement is only as trustworthy as the notion of quality it estimates, and that notion has to

be fixed before any output is inspected.

Quality in a retrieval-augmented system is not one property but several, and they can fail independently of one another. The first is groundedness. An answer should rest on the documents the retriever actually supplied, and an answer that introduces material from nowhere has failed even if the material happens to be true. The second is relevance. An answer should address the question that was asked and address it directly, so that hedging, padding and drift towards a neighbouring question all count as failures, as does supplying unrelated material where the evidence will not support a reply at all. The third is correctness. An answer should agree with what is in fact the case, which is a different demand from the first and is treated separately here for a reason given below. The fourth is obedience. Whatever the system prompt asks of the output, in length, in form, in the handling of missing evidence, should be visible in the output itself, since a prompt that is not followed cannot be said to have any effect on quality at all.

All four are properties of the text the generator produces, and that restriction is deliberate rather than incidental. The system prompt reaches the answer only through the generator, so a standard that also graded the ranking returned by the retriever would be grading something the prompt cannot touch. Each of the four is estimated by a measure taken from published work rather than invented here: groundedness by faithfulness, which asks whether every claim can be traced to a supplied passage; relevance by answer relevance, which asks whether the response addresses the question put to it; correctness by agreement with the reference answer recorded in the data; and obedience by whether the output satisfies every instruction the prompt states.

Groundedness and correctness are kept apart deliberately, because collapsing them would hide the failure this thesis is concerned with. The retrieved context is neither complete nor guaranteed to be free of error. An answer can therefore be correct while ignoring the evidence entirely, drawing instead on what the generator already knows, and it can use the evidence faithfully and still be wrong because the evidence was wrong. A system that scores well on correctness alone reveals nothing about whether retrieval was used, which is precisely the behaviour a system prompt is supposed to govern.

Answers are rarely either flawless or worthless, so a standard that only sorts them into good and bad discards most of what an experiment can observe. What is needed instead is a graded notion of shortfall, one that records how far an answer falls from the standard and in which of the four respects it does so. The four are also not equally serious, and their relative weight is a property of the setting rather than of the model. A claim that no retrieved document supports is a grave failure in a clinical assistant and a minor irritation in a consumer search tool, and any honest account of quality has to leave room for that difference to be recorded rather than assumed away. Grading the shortfall in this way settles the question of abstention without a separate rule: where a wrong answer is treated as costlier than an admission of ignorance, declining to answer becomes the better option exactly when the evidence will not support a reply, which is the long-standing trade-off between committing errors and withholding judgement described by Chow (1970).

The evaluation of this thesis therefore asks a single question. Holding the query, the knowledge base and the retrieval procedure fixed, does replacing one system prompt with another move the outputs of the system towards this standard or away from it, and does the direction of that movement survive when the retrieved evidence is degraded? A baseline prompt supplies the point of comparison, and every other condition is read against it. Because the four requirements are separable, the answer can be reported both as a single summary of overall quality and as a breakdown showing which of the four the prompt improved and which, if any, it made worse.

The formal definitions this evaluation rests on are introduced in Chapter 5. That chapter states

the set of acceptable answers precisely, gives the cost function that grades an answer's shortfall against the four requirements, fixes faithfulness, answer relevance, correctness and instruction adherence as the measures that estimate them, and defines the quantity that expresses the effect of a prompt condition relative to the baseline. It also specifies the corpus, the query set, the experimental conditions and the statistical procedure through which that quantity is estimated.

5 Experimental Setup

This chapter turns the model of Chapter 4 into a concrete procedure. Chapter 4 described the system as the distribution $p_{\text{RAG}}(y \mid q, D, S)$ of Equation (4.8) and the effect of a prompt as the intervention $\text{do}(S = S_j)$ on it. The system under test is the standard architecture, for the reasons of experimental control given in Section 2.5. The chapter describes how the corpus and the query set are assembled from the datasets of Section 4.8, which conditions the system is run under, how the parts of the cost ℓ are measured, and how the effect τ_j is estimated. All experiments are implemented in Python. Every metric used below is taken from published RAG evaluation work, and every design parameter follows a precedent named in the text. Section 5.4 names the measure used for each part of the standard that Section 4.9 set out in general terms, and Section 5.5 states that standard formally before any estimator is defined, so that no metric is chosen before the notion of quality it estimates has been fixed.

5.1 Goal and Design

The experiment manipulates exactly one component of the pipeline. The system prompt is the sole independent variable; every other part of the system is held at a fixed value for the whole of the data collection. This is a deliberate control rather than a convenience. A retrieval-augmented system has many parts that are known to move answer quality on their own, and each of them, if left free, would offer an alternative explanation for any difference observed between prompt conditions. Fixing them removes those explanations in advance, so that a difference in the measured cost can be attributed to the prompt and to nothing else.

The components held constant are the following. The generator is a single instruction-tuned language model, fixed in both its identity and its parameters θ for the whole of the data collection, and its decoding temperature and maximum output length are fixed with it, since the temperature of Equation (2.1) governs how far the sampling distribution is flattened and would otherwise be confounded with the prompt. Queries and passages are encoded by one and the same embedding model, applied to both sides of the comparison in the manner of the dense retriever of Karpukhin et al. (2020). The vector database and the index built over it, implemented with the similarity-search library of Johnson et al. (2021), are constructed once before any condition is run and are never rebuilt thereafter. The chunking strategy χ is one passage per chunk. Chen et al. (2024b) compare retrieval units of different granularity and report that finer units than the passage retrieve better, so the passage is fixed here not because it is the strongest choice but because granularity must be held constant for the prompt contrast to be interpretable, and the passage is the unit in which the source datasets are already distributed. The remaining components are likewise fixed. Because the retriever and its parameters η are determined entirely by these choices, they do not vary either. None of these is re-tuned, re-indexed or re-selected at any point during the experiment, and no condition is run against a rebuilt index.

Fixing the retrieval side has a second consequence that matters for the causal claim, and varying the depth does not weaken it. Because η , χ , the embedding model and the index are all constant,

the set $\mathcal{Z}_K(q, D)$ of Equation (4.2) depends only on the query and on the depth at which it is taken, and never on the system prompt. Within a depth level, therefore, the same passages are retrieved in every prompt condition, and every condition receives exactly the same evidence. Any change in the answer originates at the generator, which is exactly the localisation the intervention of Chapter 4 assumes. Because every prompt contrast is estimated within a depth level, the depth axis multiplies the number of cells in which that contrast is measured without entering any contrast itself.

Two further factors vary, and neither is a second treatment. The first is the quality of the evidence placed in the context, set to four levels and described in Section 5.3, because a prompt effect that appears only when the evidence is clean would be of little practical interest. The second is the retrieval depth, set to $K \in \{0, 5, 10\}$ passages per query. Depth is included for the same kind of reason as evidence quality rather than for one of its own: an instruction that governs behaviour when the context holds five passages has established little if it fails once the context holds ten, and the amount of material a prompt has to govern is exactly the sort of deployment parameter that differs between applications reusing the same pipeline. The three values are chosen to bracket that range rather than to sample it finely. $K = 5$ is the reference, the smallest depth at which the context can hold the gold evidence a multi-hop query needs together with a realistic number of distractors; $K = 10$ doubles the material without changing anything else, so that the prompt is asked to govern a longer and noisier context; and $K = 0$ supplies no retrieved passage at all and therefore coincides by construction with the empty evidence level $c^\emptyset$ of Section 5.3, which is already part of the design. The depth axis accordingly adds one new set of cells, those at $K = 10$, and folds the existing empty-context control into its lower end rather than introducing a third condition alongside it.

Both are controlled robustness factors in the same sense: the contrast of interest is always between prompt conditions, estimated separately within each level of each factor, and the levels themselves are not compared to make a claim about them. Depth is fixed before any prompt condition is applied and is never re-selected during a run, so within a level it is as constant as the components listed above; what changes is that the experiment is now run at more than one of its values. A single observation of the design is then

$$y_{ijk r} \sim p_{\text{RAG}}(y \mid q_i, \text{do}(C = c_{ik}), \text{do}(S = S_j)), \quad r = 1, \dots, n, \quad (5.1)$$

The fifteen prompt conditions and the four evidence levels are fully crossed, giving 60 cells at the reference depth. The depth axis applies only where the context is actually retrieved, that is at c^{ret} , since the other three levels set C directly and do not consult the retriever at all; running that level at $K = 10$ as well as at $K = 5$ therefore adds a further fifteen cells, and $K = 0$ adds none, being the empty level already counted. The design comprises 75 cells in all. Every query is run in every cell, so each query is its own control and no difference between conditions can be produced by a difference in the queries they were asked.

Within a cell the only remaining source of variation is the random term ε of Equation (4.11), which is why each cell is repeated $n = 5$ times. Five is the repetition budget of Zhuo et al. (2024), whose study of prompt sensitivity repeats generation under a fixed prompt so that the sensitivity being measured can be separated from the noise of sampling, and it is adopted here for three reasons rather than taken over on authority. The first concerns what a repetition buys. Each generation in a cell is a draw from the same distribution $p_{\text{RAG}}(y \mid q, D, S_j)$, so averaging n of them leaves the quantity being estimated unchanged and reduces the standard error of the cell mean by a factor of $\sqrt{n}$. Going from three draws to five is therefore not a change of estimand but a change of

resolution: it narrows the interval around every $\hat{\tau}_{jk}$ without altering what $\hat{\tau}_{jk}$ means, which matters here because the differences this experiment looks for are differences between wordings of the same requirement set and are expected to be small. The second concerns who fixes the number. A repetition count settled after inspecting the variance of the first cells would let the sample size be tuned to the effect it is meant to detect, which is a degree of freedom the experimenter should not hold in a study whose entire claim rests on having removed alternative explanations in advance; taking the count from published practice settles it before any output is seen. The third is that the choice must remain affordable at the scale the full crossing imposes, and at sixty cells and one hundred queries five repetitions is the largest budget that keeps the run tractable. Where the three pull against one another it is the second that decides, since a defensible experiment is worth more here than a marginally tighter interval. Salinas and Morstatter (2024) remove this term instead, decoding at temperature 0 so that one generation per condition suffices. That is not the setting studied here. A deployed system samples, and a prompt effect worth reporting is one that survives sampling, so the term is retained and averaged over rather than switched off. Seventy-five cells, one hundred queries and five repetitions give 37,500 generations.

5.2 Knowledge Base and Query Set

The corpus is collected, not generated. Following Section 4.8, the knowledge base D is assembled from the MS MARCO passage collection (Nguyen et al., 2016). The index holds every passage attached to a validation row whose `wellFormedAnswers` field is non-empty, $N = 122,678$ passages in total. No distractor pool is sampled: the gold passages for a query compete against the whole corpus, and whatever else the retriever ranks above them is a distractor the collection itself supplied. Because 0.38 GB of float32 vectors fits in memory at this size, the index is exhaustive rather than approximate, so retrieval depends on the query alone and not on how the index was built. Nothing in D is written for this thesis, and the annotations, not the experimenter, fix which passage answers which query.

What the empirical route gives up is the closed world. On an invented corpus the fact set Φ is complete by construction, so any claim in an output can be checked against it. Here Φ contains the annotated gold answers and the facts stated by the passages in D , which is a subset of everything true. Correctness in the sense of Equation (??) is therefore evaluated only over the claims that bear on a_q : an output asserting some further true fact absent from Φ is not counted against requirement 3. Section 5.6 states what this costs. The gain is that the retrieval judgements, the answer annotations and the queries themselves come from outside the experiment, so the difficulty of the query set was not chosen by the person testing the hypothesis, and a third party can rebuild the index from the published datasets.

Each passage forms one chunk, for the reason given in Section 5.1, and passages are used at the length at which they were published, which runs to between two and six sentences. Retrieval returns the K chunks with the highest retrieval probability, the set $\mathcal{Z}_K(q, D)$ of Equation (4.2), at each of the depths $K \in \{5, 10\}$ at which the retrieved level is run. The composition of the top five is not engineered. A direct query needs one gold passage among the five and a multi-hop query needs two or more, and whether they arrive is a property of the retriever and the corpus rather than a quantity set in advance. This is stricter than the distractor setting of Yang et al. (2018), in which two gold paragraphs are placed among eight distractors by construction, and it is what the retrieval gate of Section 5.5 exists to measure. The gate is applied separately at each depth, since a query whose gold evidence is missed at $K = 5$ may be recovered at $K = 10$. The position of the

gold passages inside the context is randomised and recorded, since Liu et al. (2024a) report that accuracy depends on where the relevant passage sits.

Because the retriever is fixed before any condition is run, how well it performs is a property of the experimental control and can be reported once, ahead of the results, and it is reported at each depth the retrieved level is run at. Against the full corpus at $K = 5$ the retriever places every gold passage a query needs inside the top five for 66 of the 100 queries, 41 of the 50 direct and 25 of the 50 multi-hop; gold recall over all annotated passages is 113/153, or 73.9%. The corresponding figures at $K = 10$ are recorded when the retrieval stage is run at that depth and are reported alongside them; recall cannot fall as the depth grows, so they bound the $K = 5$ figures from above. The gap between the categories is a consequence of the gate rather than a difference in difficulty: a direct query fails only if its single gold passage is missed, whereas a multi-hop query fails if any one of its two or more gold passages is missed, and among the 34 failures 17 retrieved one gold passage of the two they needed. These figures fix the size of the retrieved level at each depth. The 34 queries that fail the gate at $K = 5$ are excluded at c^{ret} with $K = 5$ alone, where the absence of the evidence would otherwise be confounded with the prompt; the exclusion is recomputed at $K = 10$ from that depth’s own gate, so the denominator is stated per depth and the two are not pooled. The remaining three levels set C directly and so are unaffected, and all 100 queries are used at each of them.

The query set contains $N_Q = 100$ queries, sampled from MS MARCO so that the reference answer a_q is given by annotation rather than by construction. Two categories are used, fifty queries each. They differ in one property, how far the evidence for an answer is spread across passages, and each is obtained by a documented sampling rule:

$\mathcal{Q}_{\text{direct}}$. Fifty items whose answer is carried by a single passage marked relevant for the query. One passage suffices, so a failure here is a failure to use evidence that was present. This is the noise-robustness testbed of Chen et al. (2024a), and it tests whether the system extracts evidence that is directly available.

$\mathcal{Q}_{\text{multi-hop}}$. Fifty items whose human-written answer draws on two or more of the passages marked relevant for that query. The annotation identifies which passages those are, so the requirement to combine evidence is externally recorded rather than asserted. This is the information-integration testbed, and it tests whether the system aggregates evidence distributed over several passages.

Two of the four testbeds of Chen et al. (2024a) are therefore not represented on the query axis: evidence that contradicts what the model already holds, and evidence that will not support an answer at all. Neither is abandoned. Both are properties of the context rather than of the query, and Section 5.3 varies the context across four levels independently of the category a query belongs to. The corrupted level sets the retrieved passage against the answer the model is most likely to hold in its parameters, which is the counterfactual condition; the empty level withholds evidence entirely, so that on every query the correct behaviour is to decline, which is the abstention condition. Separating the two axes in this way leaves the query set to vary how the evidence is distributed and the evidence levels to vary how good it is, rather than encoding both on the same axis.

Public datasets carry a risk an invented corpus does not: the generator may have seen the questions and their answers during pretraining. This is why the empty-context quantity π_i of Section 5.3 is a primary control here rather than a formality. The runs of each query under $c^\emptyset$, from which π_i is computed, are cells of the design already counted in the 30,000 generations rather than

an additional pass, but they are read off and fixed before the main analysis is carried out. Results are reported over the full query set and again over the subset with $\pi_i = 0$, which is the subset on which retrieval was actually tested. Chen et al. (2024a) and Longpre et al. (2021) report accuracy without documents for the same reason.

5.3 Experimental Conditions

The fifteen prompt conditions are those of Section 4.5. Each of the three requirement sets is realised in five wordings: $S_{b,0}$ through $S_{b,3}$ state the same Σ_b in four wordings, and $S_{b,4}$ states a modified set Σ'_b in which every obligation is weakened to a request. Section 5.5 reproduces the three canonical forms and gives their provenance. Each requirement is checkable on an output, in the sense of the verifiable instructions of Zhou et al. (2023b).

What is checked differs by baseline, because the baselines do not ask for the same things, and a condition is held only to what its own prompt states. All three ground the answer in the supplied documents, which is the requirement faithfulness measures. Beyond grounding, Σ_B fixes the syntax and density of citations, so adherence is checked by parsing document identifiers and counting them per sentence; Σ_C requires that the answer be justified, so adherence is checked by looking for a justification marker. Σ_A states nothing beyond grounding, so it carries no rule-checkable requirement of its own: its adherence measure is undefined by construction rather than left out by oversight, and faithfulness carries the whole of what that baseline asks for. Holding every baseline to a single common checklist would score conditions against requirements their prompts never stated, which is why no such checklist is used.

The four evidence levels are all produced by setting C directly rather than by changing the corpus and retrieving again, which is what keeps them compatible with the control described in Section 5.1: the embedding model, the index, the chunking and the retrieval depth are never touched, and no level is obtained by re-running a modified retriever. Setting C also isolates the effect on generation, since the retrieved passages enter the generator directly in Equation (4.4), whereas changing D would measure the corruption and the retriever’s reaction to it together. Each level is an intervention $\text{do}(C = c_{ik})$ in the sense of Chapter 4.

The first level, written c^{gold} , supplies only the passages the annotation marks as supporting the answer, and so records how the system behaves when the evidence is as good as it can be. The second, c^{ret} , supplies the genuine top- K output of the fixed retriever, whatever that happens to contain on a given query; it is the deployment condition, the only level at which the system is observed as it would actually run, and the only level the depth axis of Section 5.1 applies to, since it is the only one whose context comes from the retriever at all. The third, c^{corr} , takes the gold passages and negates the facts that support the answer, following the entity-substitution method of Longpre et al. (2021) and matching the counterfactual documents of Chen et al. (2024a); it sets the retrieved evidence directly against whatever the generator holds in its parameters. The substitution is performed when the context is assembled and never in the index, so the corpus is the same for this level as for every other. The level is constructed for both query categories, since the design is fully crossed, and it is the condition under which evidence is set against whatever the generator holds in its parameters. Items whose annotated answer is a single unambiguous entity admit the cleanest substitution; where an answer is longer or less definite the level functions as severe evidence corruption rather than as a sharp counterfactual. The fourth, $c^{\emptyset}$, supplies no context whatever and therefore isolates parametric knowledge on its own; it is the same cell as the retrieved level taken at depth $K = 0$, which is why the depth axis needs no separate condition at its lower end. None

of the four is compared against another to make a claim about evidence quality; each exists so that the prompt contrast can be estimated again under different conditions.

The empty level $c^\emptyset$ is the control for what the model already knows. Figure 4.1 shows θ as a cause of the answer, and without this condition its contribution cannot be separated from that of the retrieved evidence. A query with high π_i never tested retrieval. Chen et al. (2024a) report accuracy without documents alongside accuracy with counterfactual documents for the same reason, and Longpre et al. (2021) show that models often answer from memory when the context contradicts it. Results are therefore reported over the full query set and again restricted to queries with $\pi_i = 0$. Because the queries are drawn from public datasets that the generator may have seen during pretraining, a high π_i is expected for part of the query set rather than being the exception, which is why the restricted analysis carries more weight here than it would on an invented corpus.

5.4 Metrics

Each part of the standard set out in general terms in Section 4.9 is estimated by a named measure, and the scope of that measurement is fixed here before any of them is introduced. The standard is restated formally in Section 5.5, together with the cost function that grades an answer against it and the quantity that expresses the effect of a prompt condition, since that is where the analysis needs them.

Every measure used here is a generation-stage measure, and retrieval-stage measures are excluded by design. The reason is structural rather than practical. The intervention studied in this thesis sets the system prompt, and the system prompt enters the model at one place only, namely the generator, which produces the answer once the evidence has already been selected. The retrieval term of the model, as Chapter 4 sets it out, does not take the system prompt as an argument at all, so retrieval lies upstream of the intervention and cannot respond to it. Section 5.1 fixes the embedding model, the index and the chunking in addition, and holds the retrieval depth constant within each level of the depth axis, so the passages returned for a given query are not merely unaffected in expectation but literally the same in all fifteen prompt conditions of a given cell. Measures of ranking quality such as mean reciprocal rank or context precision are therefore constant across conditions by construction; a difference in them would be an arithmetic impossibility rather than a finding, and would point to a fault in the implementation. Retrieval quality is established once, as a property of the fixed pipeline, and thereafter belongs to the experimental control rather than to the results. What the experiment measures is confined to what lies downstream of the intervention: given the same evidence in every condition, what the generator does with it.

Since many different texts can express the same answer, what the standard defines is a set of acceptable answers rather than one ideal string. An answer belongs to that set when it satisfies four requirements at once:

Grounded. It asserts only what the retrieved passages state, introducing nothing from outside them.

Relevant. It addresses the question that was asked, and where the evidence will not support an answer it says so rather than producing something off-target.

Correct. It agrees with the reference answer supplied by the dataset annotation.

Obedient. It satisfies every requirement the system prompt states.

An answer that breaks none of the four is a good answer; one that breaks several is worse than one that breaks a single requirement, and a breach of a requirement that matters in the setting the

system is built for weighs more heavily than a breach of one that does not. Section 5.5 makes that grading precise.

The first three requirements are estimated by the three measures that the survey of Yu et al. (2024) names as the evaluation targets of the generation component, as distinct from those belonging to retrieval. They are adopted here rather than defined anew, and each is suited to this particular setup for a reason worth stating.

Faithfulness, also called groundedness. Whether the answer is derived exclusively from the retrieved context, so that every claim it makes can be traced to a supplied passage and nothing is introduced from elsewhere. This is the measure the experiment turns on. Every one of the fifteen prompt conditions states a grounding constraint, since all three published baselines ground the answer in the supplied documents; within each baseline four state it in identical terms and the fifth with its modal force weakened, and faithfulness is what reveals whether that constraint is obeyed in practice or merely present in the wording. Because the retrieved evidence is identical across conditions, a change in faithfulness cannot be attributed to a change in what was available to the generator, and hallucination becomes observable as a property of the prompt rather than of the retrieval. Faithfulness estimates the first component of the cost, and is scored in the reference-free manner of Es et al. (2023).

Answer relevance. Whether the answer addresses the question that was actually asked, as opposed to a neighbouring one, and whether it does so directly. A system prompt can degrade an answer without making it false. Instructions that stress caution can produce hedging and evasion, instructions that stress completeness can produce padding, and instructions that impose a format can pull the response away from the question. Relevance captures these failures, which faithfulness and correctness both miss, and it is also where abstention is read: under the empty evidence level, where no query has support in the context, an answer that declines and says why is responsive, whereas one that supplies unrelated material is not. Answer relevance estimates the second component of the cost, and is likewise among the reference-free measures of Es et al. (2023).

Correctness. Whether the answer agrees with the ground-truth answer recorded in the dataset annotation. This is the end-to-end measure of whether the system was of any use, and it is available here precisely because the query set of Section 5.2 is drawn from annotated question-answer datasets rather than constructed for the experiment. Correctness and faithfulness come apart under the corrupted evidence level: an answer that repeats a corrupted passage is faithful but wrong, and one that overrides it from memory is right but ungrounded. Reporting both is what makes that distinction visible. Correctness estimates the third component of the cost, and is evaluated as accuracy against the annotation in the sense of Chen et al. (2024a).

The fourth requirement is measured separately, because it concerns the prompt rather than the evidence. Instruction adherence is scored as prompt-level strict accuracy in the sense of Zhou et al. (2023b): a length limit is checked by counting tokens and a citation requirement by parsing document identifiers, and an answer counts as adherent only if it satisfies every requirement of the prompt at once. This measure is retained alongside the three above because the question of this thesis is whether a system prompt is followed, and the three generation measures record the consequences of following it without recording the following itself. Adherence estimates the fourth and last component of the cost. The requirement set is the same for the baseline and the three

reworded conditions, so adherence is directly comparable across those four; the fifth condition states a modified requirement set and is therefore reported separately.

These four measures are described here at a conceptual level, and deliberately so: what matters for the argument is which failure each one detects and why that failure is informative about the system prompt, not the particular arithmetic by which a score is produced. Yu et al. (2024) give the taxonomy the three are taken from, defining each as a pairwise relation between the response and something else: relevance between response and query, faithfulness between response and retrieved documents, and correctness between response and reference answer. That account is a taxonomy rather than a formulary, and the operational definitions this thesis computes with, including the decomposition of an answer into atomic claims and the rules by which those claims are aggregated into a score, are those of Es et al. (2023).

Correctness and adherence are computed by program, the first as a comparison against the annotated answer and the second by rule, which is the design principle of the verifiable instructions of Zhou et al. (2023b). Faithfulness and answer relevance are scored by a separate language model, as in Es et al. (2023) and Saad-Falcon et al. (2024).

5.4.1 The Judge

Faithfulness and answer relevance are scored by a separate language model, as Chen et al. (2024a) do for comparable measures. Since this thesis assumes that language models are sensitive to their instructions, the judge cannot be assumed reliable. Three precautions are taken, following Zheng et al. (2023), who document position and verbosity biases in this setting. The judge’s agreement with human labelling is measured on about 100 held-out outputs before the main run and reported as Cohen’s kappa. Agreement is reported for each condition separately rather than pooled, because the format condition is recognisable from its surface alone and any judge error that varied with condition would bias the estimated effect. The judge’s own prompt is fixed, and outputs are shown in random order without condition labels.

5.5 Procedure and Analysis

The experiment runs in two stages. Retrieval is executed first, once for every query in the set, and its output is written to disk together with the identifiers and positions of the passages returned. Generation is executed second, reading that stored output rather than calling the retriever again. Every generation is an independent request with caching disabled, and the query, the condition indices, the context string, the output and the number of output tokens are recorded for each one.

The separation is not an implementation convenience. Section 4.6 assumes that the system prompt has no effect on which documents are retrieved, and running retrieval to completion before any prompt condition is applied is what enforces that assumption rather than merely asserting it. No execution path remains along which a prompt could reach the retriever, because every prompt condition reads the same stored passages.

The retrieval stage is also where the evidence is checked, and it is run once per depth. A query counts as a retrieval success at a given depth when the passages its annotation marks as necessary appear among the K returned at that depth. For a direct query that is the single passage carrying the answer; for a multi-hop query it is every passage the annotated answer draws on, since an answer that requires two passages cannot be produced from one. Success is recorded per query before any output is generated.

Queries whose retrieval failed are excluded from the evaluation of the generation stage at the retrieved evidence level c^{ret} . The measures of Section 5.4 are not defined on evidence that was never supplied. When the context lacks the passage carrying the answer, a correct answer can only have come from parametric memory, which is the behaviour the grounding constraint exists to suppress, and an incorrect answer cannot be told apart from a faithful reading of the distractors. Scoring such a cell would measure the retriever and report the result as an effect of the prompt.

The exclusion is applied uniformly across prompt conditions and separately within each depth. Retrieval is identical in all fifteen conditions at a given depth, so retrieval success is a property of the query and the depth rather than of the condition, and a failed query is dropped from all fifteen at once. The pairing of Section 5.1 therefore survives: every comparison is still made within a query, and no condition is evaluated on a subset that another condition was not.

The gate applies to c^{ret} alone, at each depth that level is run at. The other three evidence levels of Section 5.3 set the context directly rather than retrieving it, so retrieval cannot fail in them: c^{gold} supplies the annotated passages by construction, c^{corr} supplies their corrupted variants, and $c^{\emptyset}$ supplies nothing by design. Those levels are evaluated over the full query set.

Two consequences follow for reporting. The number of retrieval successes is stated with the results at each depth, since it fixes the denominator of every average taken at c^{ret} , and that denominator is smaller than the one used at the other levels and differs between depths. And retrieval success is reported as a property of the fixed pipeline in the sense of Section 5.4: it gates which cells are interpretable, and is not an outcome the prompt is credited or blamed for.

5.5.1 Provenance of the Prompt Wordings

The prompt conditions are not composed from scratch. Their wording is taken from prompts published with retrieval-augmented systems, so that the baseline is a form already in use rather than one written to be improved upon. Three published prompts bear on the requirement set of Section 5.3, and each is reproduced below as it appears in its source. They differ in how many requirements they state, which is the property that matters here: a prompt stating two requirements offers little for a change in wording to act on, while one stating ten offers a great deal.

The response-generation prompt of Friel et al. (2024) states the fewest. It grounds the answer in the supplied documents and asks the question, and does nothing else.

```
Use the following pieces of context to answer the question.
{documents}
Question: {question}
```

Listing 5.1: The response-generation prompt of Friel et al. (2024).

The instruction of the VANILLA configuration of Gao et al. (2023b) states the most. Beyond grounding, it constrains accuracy, engagement, concision, register, and the syntax and density of citations, and it warns that some supplied results may be irrelevant.

```
Write an accurate, engaging, and concise answer for the given
question using only the provided search results (some of which
might be irrelevant) and cite them properly. Use an unbiased and
journalistic tone. Always cite for any factual claim. When citing
several search results, use [1][2][3]. Cite at least one document
and at most three documents in each sentence. If multiple documents
support the sentence, only cite a minimum sufficient subset of the
```

```
documents.
```

Listing 5.2: The VANILLA instruction of Gao et al. (2023b), reproduced from their Table 23.

Between the two sits the instruction of Mao et al. (2024), which is the one direct precedent for the decomposition of Equation (4.9). Its authors describe it as consisting of three parts and give a rationale for each: the first directs the model to the retrieved passages, the second asks it to read the question carefully, and the third asks for an answer together with a justification. Writing a prompt as an enumerated set of requirements with a stated purpose per requirement is exactly what $\rho(\Sigma, \varphi)$ formalises, and Mao et al. (2024) report that this form outperforms both a bare instruction and a chain-of-thought variant on the same passages.

```
Refer to the passage below and answer the following question.  
Make sure you fully understand the meaning of the question and  
passages. Then give the answer and explain why you choose this  
answer.  
Passages: {passages}  
Question: {question}
```

Listing 5.3: The comprehensive instruction of Mao et al. (2024).

Two points follow for the conditions of Section 5.3. All three sets are used rather than one at an intermediate density, because the objection each would face alone is answered only by the others: a set stating a single requirement leaves the deontic condition almost nothing to weaken, while a set stating ten risks confounding the wording manipulation with the sheer length of the prompt. Observing the same wording effect across all three separates the effect of wording from the effect of density. And no requirement checked here is an invention of this thesis: the citation constraint is the central requirement of Gao et al. (2023b) and the justification constraint the third part of Mao et al. (2024), which is why adherence to each can be checked by parsing the output rather than by judgement.

5.6 Limitations

The empirical evaluation rests on a single collection. Every result reported here is conditional on MS MARCO: on its queries, on the passages it supplies, on the relevance judgements its annotators made, and on the answers they wrote. Section 4.8 argues that the topical spread within that collection is wide, and the argument holds, but spread within one corpus is not replication across several. A prompt effect established here bounds what may be expected elsewhere rather than establishing it.

The composition of the collection sets a second bound. MS MARCO records what people type into a web search engine, so its queries are short, largely factoid, and drawn from the domains web search covers. Several of the deployments that motivate retrieval in Section 2.4, clinical decision support and legal question answering among them, are absent from it, and the documents those settings retrieve over differ from web passages in length, structure and register. Whether an instruction that governs behaviour on this collection governs it on those is left to future work.

6 Results and Discussion

7 Conclusion

7.1 Contribution

7.2 Future Work

Bibliography

- Abootorabi, M. M., Zobeiri, A., Dehghani, M., Mohammadkhani, M., Mohammadi, B., Ghahroodi, O., Baghshah, M. S., and Asgari, E. (2025). Ask in any modality: A comprehensive survey on multimodal retrieval-augmented generation.
- Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*.
- Bush, V. (1945). As we may think. *The Atlantic Monthly*, 176(1):101–108.
- Cheerla, C. (2025). Advancing retrieval-augmented generation for structured enterprise and internal data.
- Chen, J., Lin, H., Han, X., and Sun, L. (2024a). Benchmarking large language models in retrieval-augmented generation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38:17754–17762.
- Chen, T., Wang, H., Chen, S., Yu, W., Ma, K., Zhao, X., Zhang, H., and Yu, D. (2024b). Dense X retrieval: What retrieval granularity should we use? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 15159–15177.
- Cheng, Z. and Mastropaolo, A. (2026). An empirical study on the effects of system prompts in instruction-tuned models for code generation.
- Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46.
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387.
- Cormack, G. V., Clarke, C. L. A., and Büttcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization.
- Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2023). Ragas: Automated evaluation of retrieval augmented generation.
- Feder, A., Keith, K. A., Manzoor, E., Pryzant, R., Sridhar, D., Wood-Doughty, Z., Eisenstein, J., Grimmer, J., Reichart, R., Roberts, M. E., Stewart, B. M., Veitch, V., and Yang, D. (2022). Causal inference in natural language processing: Estimation, prediction, interpretation and beyond. *Transactions of the Association for Computational Linguistics*, 10:1138–1158.

- Friel, R., Belyi, M., and Sanyal, A. (2024). RAGBench: Explainable benchmark for retrieval-augmented generation systems.
- Gao, L., Ma, X., Lin, J., and Callan, J. (2023a). Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pages 1762–1777.
- Gao, T., Yen, H., Yu, J., and Chen, D. (2023b). Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2023c). Retrieval-augmented generation for large language models: A survey.
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*.
- He, J., Zhang, B., Rouhizadeh, H., Chen, Y., Yang, R., Lu, J., Chen, X., Liu, N., and Teodoro, D. (2025). Retrieval-augmented generation in biomedicine: A survey of technologies, datasets, and clinical applications.
- Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*.
- Hua, A., Tang, K., Gu, C., Gu, J., Wong, E., and Qin, Y. (2025). Flaw or artifact? rethinking prompt sensitivity in evaluating LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.
- Jeong, S., Baek, J., Cho, S., Hwang, S. J., and Park, J. C. (2024). Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Volume 1: Long Papers*, pages 7036–7050.
- Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S., Wang, D., Zamani, H., and Han, J. (2025). Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning.
- Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics.
- Kostić, B., Fallon, C., Risch, J., and Löser, A. (2026). Same meaning, different scores: Lexical and syntactic sensitivity in LLM evaluation. To appear in the Proceedings of the 2026 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC).
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.

- Lin, Y.-H., Chen, Q.-H., Cheng, Y.-J., Zhang, J.-R., Liu, Y.-H., Hsia, L.-Y., and Chen, Y.-N. (2025). LLM inference enhanced by external knowledge: A survey.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024a). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Liu, Z., Ping, W., Roy, R., Xu, P., Lee, C., Shoeybi, M., and Catanzaro, B. (2024b). ChatQA: Surpassing GPT-4 on conversational QA and RAG. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.
- Longpre, S., Perisetla, K., Chen, A., Ramesh, N., DuBois, C., and Singh, S. (2021). Entity-based knowledge conflicts in question answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7052–7063.
- Lu, S., Schuff, H., and Gurevych, I. (2024). How are prompts different in terms of sensitivity? In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 5833–5856.
- Malkov, Y. A. and Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836.
- Mao, Y., Dong, X., Xu, W., Gao, Y., Wei, B., and Zhang, Y. (2024). FIT-RAG: Black-box RAG with factual information and token reduction.
- Mu, N., Lu, J., Lavery, M., and Wagner, D. A. (2025). A closer look at system prompt robustness.
- Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., and Deng, L. (2016). MS MARCO: A human generated MACHine Reading COMprehension dataset.
- Niu, C., Wu, Y., Zhu, J., Xu, S., Shum, K., Zhong, R., Song, J., and Zhang, T. (2024). RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*.
- Oche, A. J., Folashade, A. G., Ghosal, T., and Biswas, A. (2025). A systematic review of key retrieval-augmented generation (RAG) systems: Progress, gaps, and future directions.
- Ovadia, O., Brief, M., Mishaeli, M., and Elisha, O. (2024). Fine-tuning or retrieval? comparing knowledge injection in LLMs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 237–250.
- Pan, J. J., Wang, J., and Li, G. (2024). Survey of vector database management systems. *The VLDB Journal*, 33(5):1591–1615.
- Pandya, H. A. and Bhatt, B. S. (2021). Question answering survey: Directions, challenges, datasets, evaluation matrices.
- Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2 edition.
- Pipitone, N. and Alami, G. H. (2024). LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain.

- Qin, R., Wang, Q., Wang, T., Wei, Z., and Shen, W. (2026). Evaluating and explaining prompt sensitivity of LLMs using interactions.
- Robertson, S. and Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.
- Ruiz, F. J. and Cardona-Betancourt, V. (2026). Prompt carefully! ChatGPT displays rule-based insensitivity to contingencies. *Journal of Contextual Behavioral Science*, 41:101012.
- Saad-Falcon, J., Khattab, O., Potts, C., and Zaharia, M. (2024). ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 338–354.
- Salinas, A. and Morstatter, F. (2024). The butterfly effect of altering prompts: How small changes and jailbreaks affect large language model performance. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- Salton, G., Wong, A., and Yang, C. S. (1975). A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620.
- Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. (2024). Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*.
- Singh, A., Ehtesham, A., Kumar, S., Khoei, T. T., and Vasilakos, A. V. (2025). Agentic retrieval-augmented generation: A survey on agentic RAG.
- Wahle, J. P., Ruas, T., Xu, Y., and Gipp, B. (2024). Paraphrase types elicit prompt engineering capabilities. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 11004–11033, Miami, Florida, USA. Association for Computational Linguistics.
- Wang, M., Yao, Y., Xu, Z., Qiao, S., Deng, S., Wang, P., Chen, X., Gu, J.-C., Jiang, Y., Xie, P., Huang, F., Chen, H., and Zhang, N. (2024). Knowledge mechanisms in large language models: A survey and perspective.
- Yan, S.-Q., Gu, J.-C., Zhu, Y., and Ling, Z.-H. (2024). Corrective retrieval augmented generation.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2369–2380.
- Ying, J., Cao, Y., Xiong, K., Cui, L., He, Y., and Liu, Y. (2024). Intuitive or dependent? investigating LLMs’ behavior style to conflicting prompts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, pages 4221–4246.
- Yu, H., Gan, A., Zhang, K., Tong, S., Liu, Q., and Liu, Z. (2024). Evaluation of retrieval-augmented generation: A survey.

- Zhan, P., Xu, Z., Tan, Q., Song, J., and Xie, R. (2024). Unveiling the lexical sensitivity of LLMs: Combinatorial optimization for prompt enhancement. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5128–5154, Miami, Florida, USA. Association for Computational Linguistics.
- Zhang, B., Xin, H., Li, J., Zhang, D., Fang, M., Liu, Z., Nie, L., and Liu, Z. (2025). Benchmarking poisoning attacks against retrieval-augmented generation.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
- Zheng, M., Pei, J., Logeswaran, L., Lee, M., and Jurgens, D. (2024). When “a helpful assistant” is not really helpful: Personas in system prompts do not improve performances of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15126–15154.
- Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., and Levy, O. (2023a). LIMA: Less is more for alignment. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*.
- Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. (2023b). Instruction-following evaluation for large language models.
- Zhuo, J., Zhang, S., Fang, X., Duan, H., Lin, D., and Chen, K. (2024). ProSA: Assessing and understanding the prompt sensitivity of LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*.